

PROMPT TO PROFIT™

WORKBOOK 06

Sales Tracker

Build a Complete Sales Tracking Application
with AI

TECHNOLOGY STACK

HTML

CSS

Vanilla JavaScript

Supabase

DIFFICULTY

Beginner

ESTIMATED COMPLETION TIME

20-25 Hours

PRODUCED BY

Tochukwu Tech and AI Academy

www.tochukwunkwocha.com

PROMPT TO PROFIT™

Workbook 06 · Sales Tracker

© 2026 Tochukwu Tech and AI Academy. All rights reserved.

Produced and published by Tochukwu Tech and AI Academy.

www.tochukwunkwocha.com

This workbook is designed for personal learning and guided software-building practice.

Technology names and product names remain the property of their respective owners.

Contents

Workbook 06 · Sales Tracker

LEARNER SUPPORT TOOLKIT	4
VISUAL GLOSSARY	5
WORK SAFELY: MAKE A BACKUP	7
SOMETHING WENT WRONG – WHAT SHOULD I CHECK?	8
MY ERROR LOG	10
CHAPTER 1 · BUILDING THE PUBLIC WEBSITE	12
LESSON 1 · UNDERSTANDING THE SALES TRACKER	13
LESSON 2 · BUILDING THE LANDING PAGE	16
LESSON 3 · STYLING THE LANDING PAGE	30
LESSON 4 · ADDING THE RESPONSIVE NAVIGATION	41
LESSON 5 · COMPLETING THE PUBLIC WEBSITE	50
CHAPTER SUMMARY	59
CHAPTER MILESTONE	60
CHAPTER 2 · CONNECTING TO SUPABASE	62
LESSON 1 · CREATING YOUR SUPABASE PROJECT	64
LESSON 2 · UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY	69
LESSON 3 · CREATING YOUR SUPABASE CONNECTION FILE	74
LESSON 4 · BUILDING A CONNECTION TEST PAGE	81
LESSON 5 · FINAL CONNECTION REVIEW	89
CHAPTER SUMMARY	94
CHAPTER MILESTONE	95
CHAPTER 3 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	96
LESSON 1 · UNDERSTANDING THE AUTHENTICATION FLOW	99
LESSON 2 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	103
LESSON 3 · AUDITING THE AUTHENTICATION SYSTEM	125

CHAPTER SUMMARY	130
CHAPTER MILESTONE	131
CHAPTER 4 · BUILDING THE SALES DATABASE	132
LESSON 1 · UNDERSTANDING THE SALES DATA MODEL	133
LESSON 2 · CREATING THE SALES TABLE	137
LESSON 3 · PROTECTING SALES VALUES	141
LESSON 4 · ENABLING ROW LEVEL SECURITY	145
LESSON 5 · CREATING THE SELECT POLICY	149
LESSON 6 · CREATING THE INSERT POLICY	153
LESSON 7 · CREATING THE UPDATE POLICY	157
LESSON 8 · CREATING THE DELETE POLICY	161
LESSON 9 · TESTING THE SALES DATABASE	165
CHAPTER SUMMARY	169
CHAPTER MILESTONE	170
CHAPTER 5 · BUILDING THE SALES DASHBOARD	171
LESSON 1 · DESIGNING THE SALES DASHBOARD	172
LESSON 2 · BUILDING THE RECORD SALE FORM	176
LESSON 3 · SAVING SALES SECURELY	180
LESSON 4 · BUILDING DAILY AND MONTHLY STATISTICS	184
CHAPTER SUMMARY	188
CHAPTER MILESTONE	189
CHAPTER 6 · VIEWING SALES	190
LESSON 1 · BUILDING THE SALES HISTORY	191
LESSON 2 · BUILDING THE SALE DETAILS PAGE	195
LESSON 3 · TESTING THE SALES HISTORY	199
CHAPTER SUMMARY	202
CHAPTER MILESTONE	203
CHAPTER 7 · SEARCHING, FILTERING AND SORTING SALES	204
LESSON 1 · BUILDING SALES SEARCH, FILTERING AND SORTING	205
LESSON 2 · TESTING SALES DISCOVERY	209
CHAPTER SUMMARY	213

CHAPTER MILESTONE	214
CHAPTER 8 • EDITING SALES	215
LESSON 1 • BUILDING SALE EDITING	216
LESSON 2 • TESTING SALE EDITING	220
CHAPTER SUMMARY	224
CHAPTER MILESTONE	225
CHAPTER 9 • DELETING SALES	226
LESSON 1 • BUILDING SECURE SALE DELETION	227
LESSON 2 • TESTING SALE DELETION	231
CHAPTER SUMMARY	235
CHAPTER MILESTONE	236
CHAPTER 10 • FINAL APPLICATION REVIEW	237
LESSON 1 • COMPLETE SYSTEM TESTING	238
CHAPTER SUMMARY	242
CHAPTER MILESTONE	243
REFLECTION QUESTIONS	245
EXTENSION CHALLENGES	246
NEXT WORKBOOK	247

ABOUT THIS WORKBOOK

This workbook is a complete, self-contained project.

You do not need to own any of the other Prompt to Profit™ Software Workbooks to complete it successfully.

Every workbook in this series teaches you how to build one complete software application from start to finish. It contains all the explanations, prompts, instructions and practical exercises you need to complete that project independently.

Although some software development concepts naturally appear across multiple workbooks, every workbook starts from the beginning of its own project and assumes no prior knowledge of the other workbooks.

This means you can study the workbooks in any order that suits your goals.

If your immediate need is to build a Sales Tracker, you can begin with this workbook.

If you later decide to build additional software, each new workbook will teach you a different business application while reinforcing the software development skills you have already learned.

Together, the workbooks form a complete software development series. Individually, each workbook is a complete learning experience.

WELCOME

Welcome to Prompt to Profit™ Workbook 06.

In this workbook, you will build a complete Sales Tracker from the ground up using HTML, CSS, Vanilla JavaScript, Supabase and Artificial Intelligence.

Throughout this project, you will learn how to build secure, database-driven business software by following the same step-by-step development process used in real-world software projects.

You will create a professional sales management application that allows businesses to register users, record sales securely, search and organise sales, update sale details, and protect every user's data using Row Level Security.

Every feature is built one step at a time. You won't simply copy prompts into an AI tool. You'll learn why each feature is being built, how the different parts of the application work together, and how to communicate effectively with AI to build reliable software.

By the end of this workbook, you will have a fully functional Sales Tracker that you can proudly include in your software development portfolio.

You will build the project using:

- HTML

- CSS
- Vanilla JavaScript
- Supabase
- ChatGPT
- Notepad
- A web browser

You do not need to install additional software.

You will continue working with complete files and clear Build Prompts.

Every new feature will preserve everything already built.

Complete the lessons in order.

Test each feature before moving forward.

Do not continue when an important feature is not working.

WHAT YOU WILL BUILD

By the end of this workbook, your Sales Tracker will include:

- A clear public website
- User registration and login
- A protected sales dashboard
- A form for recording sales
- Automatically calculated totals
- Daily, monthly and all-time sales statistics
- A complete Sales History page
- Sale Details and editing
- Search, filters and sorting
- Secure deletion
- Supabase Row Level Security
- A live HTTPS deployment

WORKBOOK STRUCTURE

This workbook is organised into ten chapters.

Chapter 1

Building the Public Website

Chapter 2

Connecting to Supabase

Chapter 3

Building the Complete Authentication System

Chapter 4

Building the Sales Database

Chapter 5

Building the Sales Dashboard

Chapter 6

Viewing Sales

Chapter 7

Searching, Filtering and Sorting Sales

Chapter 8

Editing Sales

Chapter 9

Deleting Sales

Chapter 10

Final Testing and Project Completion

Complete each chapter before moving to the next.

Every chapter depends on the work completed earlier.

LEARNER SUPPORT TOOLKIT

Keep this part of the workbook close while you build.

It explains common words, shows you how to protect your work, gives you a simple way to investigate problems, and provides a place to record errors and solutions.

You are not expected to remember everything immediately.

Return to these pages whenever you need them.

VISUAL GLOSSARY

HTML	The file that gives a web page its content and structure.
CSS	The file that controls colours, spacing, layout and appearance.
VANILLA JAVASCRIPT	JavaScript used without a framework. It controls what the application does.
BROWSER	The program used to open and test the application, such as Chrome or Edge.
FILE	One saved part of the project, such as index.html or dashboard.js.
FOLDER	The place where all files for one project are kept together.
FUNCTION	A named set of instructions that performs one task.
EVENT	An action the browser can notice, such as a click, form submission or key press.
VARIABLE	A named place used to hold information while the application is running.
URL	The address of a page, website or online service.
SUPABASE	The online service used for the database, authentication and security.
DATABASE	An organised place where the application stores information.
TABLE	A named part of the database that stores one type of information.
RECORD	One complete item saved inside a database table.
AUTHENTICATION	The process of creating accounts, signing in and identifying the current user.
SESSION	The saved sign-in state that allows the application to remember the current user.
ROW LEVEL SECURITY	Supabase rules that control which database records each user can access.

PUBLISHABLE KEY	The Supabase project key that browser-based applications are allowed to use.
DEPLOY	To place the complete project online so it has a live HTTPS website address.
DEBUGGING	The careful process of finding, understanding and correcting a problem.

WORK SAFELY: MAKE A BACKUP

A backup is a separate copy of your complete project folder.

Create a backup before replacing files that already contain working code.

1.

Close any project files that are open in Notepad.

2.

Find your complete project folder.

3.

Copy the folder.

4.

Paste the copy outside the working project folder.

Do not place the backup inside the folder you deploy.

5.

Rename the copy clearly.

For example:

Project Backup - Before Chapter 3 Lesson 2

6.

Open the backup folder.

Confirm that the expected files are inside it.

If a new change causes a serious problem, you can return to this working copy.

SOMETHING WENT WRONG — WHAT SHOULD I CHECK?

Problems are a normal part of building software.

Do not replace random pieces of code and do not continue to the next lesson.

Work through this checklist in order.

- ☐ Confirm that you opened the correct project folder.
- ☐ Confirm that every updated file was saved in Notepad.
- ☐ Check every filename carefully.
- ☐ Make sure an HTML, CSS or JavaScript file does not end with .txt.
- ☐ Confirm that you pasted the complete file returned by AI.
- ☐ Confirm that you did not paste an explanation, a code-block label or only part of a file.
- ☐ Check that stylesheet and JavaScript filenames match the names used inside the HTML.
- ☐ On Supabase pages, check that the scripts load in the order taught in the lesson.
- ☐ Read the exact message shown on the page.
- ☐ Open the browser Console and look for the first red error.

To open the Console:

Right-click the page.

Select:

Inspect

Then select:

Console

- ☐ Copy the complete first red error into your error log.
- ☐ Think about the last file you changed before the problem appeared.
- ☐ Compare that file with the lesson requirements.
- ☐ If necessary, restore the most recent working backup.

When asking AI for help, provide:

-
- What you were trying to do
 - What you expected to happen
 - What actually happened
 - The complete error message
 - The names of the files involved

Ask AI to return complete corrected files.

Do not ask for snippets.

MY ERROR LOG

Use this page whenever something does not work.

Writing down the problem helps you investigate it carefully and remember the solution.

ERROR 1

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

ERROR 2

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

CHAPTER INTRODUCTION

Every useful business application begins with a clear purpose.

A Sales Tracker helps a business record what it sells and understand its daily and monthly activity.

Before building the private sales area, you will create a simple public website that explains the problem and introduces the solution.

This chapter begins gently. You will create the project folder, build the landing page, style it and add responsive navigation.

You will continue using only HTML, CSS, Vanilla JavaScript, Notepad and a web browser.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A clear public landing page
- A hero section for the Sales Tracker
- A business problem section
- A features section
- A simple How It Works section
- A sample sales dashboard preview
- Login and Register links
- Responsive navigation

By the end of the chapter, your project will have a complete public website ready to connect to Supabase.

LESSON 1

UNDERSTANDING THE SALES TRACKER

Estimated Time

10–15 Minutes

WHAT YOU ARE BUILDING

A clear plan for the Sales Tracker and a separate folder for the new project.

WHY THIS MATTERS

Understanding the business problem first will help every later page, prompt and test make sense.

BEFORE YOU CONTINUE

- ☒ You have a computer with Notepad and a web browser.
- ☒ You are ready to keep this project separate from any other workbook.
- ☒ You understand that no code is required in this first lesson.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to generate with AI in this lesson.

Create a new empty folder named:

Sales Tracker

Then read the project plan below and make sure you understand it.

The application will allow a signed-in business user to record a sale with a date, item name, category, quantity, unit price, payment method, optional customer name and optional notes.

It will calculate totals, show daily and monthly figures, display sales history, find records, edit records and delete unwanted records.

Each account will see only its own sales.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- Nothing in this lesson.
- This lesson prepares you and your project folder for the build.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Create and save the empty Sales Tracker project folder.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open the folder and confirm it is empty.
- ☒ Explain in your own words what one saved sale will contain.
- ☒ List the seven main capabilities you will build.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ The folder is named Sales Tracker.

- ✓ The folder contains no files from another project.
- ✓ You understand that each user will have private sales.

COMMON BEGINNER MISTAKES

- Placing files from several projects in one folder.
- Starting with code before understanding the project.
- Thinking that a sale total should be typed instead of calculated.

BEHIND THE SCENES

The public website, authentication, sales database, dashboard and record-management pages will be built in a careful order. Each part will connect to the parts completed before it.

THINK LIKE A SOFTWARE DESIGNER

Which two saved numbers will the application use to calculate the total for one sale?

WHAT YOU LEARNED

You learned:

- What the Sales Tracker will do.
- What information one sale will contain.
- Why the project needs its own folder.
- Why every user's sales must remain private.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 2

BUILDING THE LANDING PAGE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the first version of the public website for your Sales Tracker.

The landing page will explain what the software does, the business problems it solves, and the main features users will receive.

At this stage, the page will contain the complete HTML structure.

It will not look polished yet because the stylesheet has not been created.

The responsive navigation will also not work until the JavaScript file is added in a later lesson.

WHY THIS MATTERS

A business software application needs a clear public website.

Before someone creates an account, they should be able to understand:

- What the software does
- Who it is designed for
- What problems it solves
- What features are available
- What action they should take next

A well-structured landing page helps visitors understand the value of the application before they register.

BEFORE YOU CONTINUE

Confirm that your project folder is named:

Sales Tracker

Open the folder.

It should still be empty.

You are about to create the first file:

index.html

When saving the file in Notepad:

- Click File.
- Click Save As.
- Open the Sales Tracker folder.
- Change Save as type to All Files.
- Enter the filename exactly as:

index.html

Make sure the file is not saved as:

index.html.txt

BUILD PROMPT

PROMPT STARTS HERE

Create one complete HTML5 file named:

index.html

The file is the public landing page for a Sales Tracker designed for small businesses.

Return the complete index.html file only.

Do not return explanations.

Do not return snippets.

Do not return CSS.

Do not return JavaScript.

Do not omit any section.

FILE CONNECTIONS

Inside the head section, include this stylesheet link exactly:

```
<link rel="stylesheet" href="styles.css">
```

Immediately before the closing body tag, include this JavaScript link exactly:

```
<script src="script.js"></script>
```

The files styles.css and script.js will be created in later lessons.

Do not include CSS inside index.html.

Do not include JavaScript inside index.html.

GENERAL HTML REQUIREMENTS

- Use proper HTML5 structure.
- Include the viewport meta tag.
- Include an appropriate page title.
- Use semantic HTML elements.
- Use clear and meaningful class names.
- Use section IDs that match the navigation links.
- Keep the content professional and easy to understand.
- Do not use external images.
- Do not use external icons.
- Do not use any framework.
- Do not use placeholder text such as Lorem Ipsum.

PAGE STRUCTURE

Build the sections below in this exact order.

1. HEADER AND NAVIGATION

Create a professional header.

Include a clickable logo that displays:

Sale Records

The logo must link to:

index.html

Include these navigation links:

Home

Features

How It Works

Login

Register

The links should point to:

Home:

#home

Features:

#features

How It Works:

#how-it-works

Login:

login.html

Register:

register.html

Style-related classes should already be added so CSS can later make Login a secondary action and Register the main action.

Add a mobile hamburger button.

The hamburger button must:

- Use a button element.
- Include exactly three empty span elements.
- Include an aria-label that explains its purpose.
- Include:

aria-expanded="false"

- Include:

aria-controls

pointing to the navigation menu.

Do not build the hamburger lines using CSS pseudo-elements.

The three span elements must exist inside the HTML.

2. HERO SECTION

Give the section the ID:

home

Include:

- A clear headline explaining that the software helps businesses organise sales information.
- A short supporting paragraph.
- A primary button labelled:

Get Started

The button must link to:

register.html

- A secondary button labelled:

See How It Works

The button must link to:

#how-it-works

Also include a sales dashboard preview built entirely with HTML.

The preview should contain simple cards or panels representing:

- Total Sales
- Today's Sales
- This Month's Sales
- A short sale list

Do not use an image.

3. BUSINESS PROBLEM SECTION

Create a section explaining the problems businesses face when sales information is scattered across:

- Notebooks
- Spreadsheets
- Messaging applications
- Loose paper records

Explain that scattered records can lead to:

- Missing sales records
- Incorrect totals
- Slow monthly reporting
- Difficulty checking daily performance
- Difficulty understanding sales performance

Keep the wording clear and concise.

4. FEATURES SECTION

Give the section the ID:

features

Include six feature cards.

The cards should explain:

1. Secure Sales Records
2. Automatic Sale Totals
3. Daily and Monthly Statistics
4. Sales History and Details
5. Secure Editing and Deletion
6. Search, Filters and Sorting

Each card should include:

- A short title
- A brief explanation

Do not use external icons.

You may use simple text labels or small decorative HTML elements.

5. HOW IT WORKS SECTION

Give the section the ID:

how-it-works

Explain the process in four clear steps:

1. Create an Account
2. Record a Sale

3. View Sales Performance

4. Find and Manage Sales

Each step should have:

- A number
- A heading
- A short explanation

6. DASHBOARD PREVIEW SECTION

Create a larger dashboard preview using HTML only.

Include:

- Four summary cards
- A sales search area
- Filter controls
- Three sample sale cards

Each sample sale card should display example information such as:

- Sale date
- Item name
- Category
- Quantity
- Total amount
- Payment method
- View Sale button

The sample data must be clearly fictional.

Do not use real personal information.

The buttons are only part of the visual preview and do not need to work.

7. FINAL CALL-TO-ACTION SECTION

Encourage the visitor to organise sales in one secure system.

Include a button labelled:

Create Your Account

The button must link to:

```
register.html
```

8. FOOTER

Include:

- Logo or software name
- A short description
- Quick links
- Login link pointing to login.html
- Register link pointing to register.html
- Copyright text

Use a JavaScript-friendly span with an ID for the copyright year if appropriate, but do not include JavaScript inside the page.

ACCESSIBILITY REQUIREMENTS

- Use heading levels in a logical order.
- Add descriptive text to links and buttons.
- Use aria-labels where necessary.
- Make the hamburger button accessible.
- Use meaningful link text.
- Avoid empty links.

IMPORTANT

The Login links must point to:

```
login.html
```

The Register and Get Started links must point to:

```
register.html
```

The stylesheet link must be:

```
<link rel="stylesheet" href="styles.css">
```

The JavaScript link must be:

```
<script src="script.js"></script>
```

Return one complete index.html file from beginning to end.

Do not include unfinished comments.

Do not return any other file.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

index.html

The file should contain:

- ✓ Correct HTML5 structure
- ✓ Stylesheet link to styles.css
- ✓ JavaScript link to script.js
- ✓ Header and navigation
- ✓ Three-span hamburger button
- ✓ Hero section
- ✓ Business problem section
- ✓ Six feature cards
- ✓ Four-step How It Works section
- ✓ Dashboard preview
- ✓ Final call-to-action
- ✓ Footer
- ✓ Login links pointing to login.html
- ✓ Register links pointing to register.html

The response should begin with:

<!DOCTYPE html>

It should not begin with an explanation.

SAVE YOUR FILES

Copy the complete HTML code returned by ChatGPT.

Open Notepad.

Paste the complete code into the empty document.

Click File.

Click Save As.

Open your Sales Tracker folder.

Choose:

Save as type: All Files

Enter the filename exactly as:

index.html

Click Save.

Close Notepad.

Open the project folder and confirm that the file appears as:

index.html

Make sure it is not named:

index.html.txt

TEST YOUR WORK

Open your Sales Tracker folder.

Double-click:

index.html

The page should open in your browser.

It will look plain because styles.css does not exist yet.

That is expected.

Scroll through the complete page.

Confirm that you can see:

☒ Header

☒ Navigation

- ☒ Hero section
- ☒ Business problem section
- ☒ Features section
- ☒ How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer

Click:

Features

The browser should move to the Features section.

Click:

How It Works

The browser should move to the How It Works section.

Click:

Login

The browser may show that login.html does not exist.

This is expected.

Click:

Register

The browser may show that register.html does not exist.

This is also expected.

Those pages will be created later as part of the complete authentication system.

CHECKPOINT

Before moving on, confirm that:

- ☒ index.html exists.
- ☒ The browser opens the file as a webpage.

- ✓ The page does not display HTML code as plain text.
- ✓ Every required section is present.
- ✓ The hamburger button contains three span elements.
- ✓ Login points to login.html.
- ✓ Register points to register.html.
- ✓ styles.css is linked.
- ✓ script.js is linked.
- ✓ The filename is not index.html.txt.

COMMON BEGINNER MISTAKES

A common mistake is saving the file using the wrong extension.

If the file is saved as:

index.html.txt

the browser may not treat it as a webpage.

Open the project folder and check the complete filename carefully.

Another mistake is removing the links to styles.css and script.js because those files do not exist yet.

Do not remove them.

They are being linked now so the files will connect correctly when they are created.

Some students also change:

login.html

to:

login

or change:

register.html

to:

signup.html

Do not do this.

The filenames must remain consistent because later pages and redirects will depend on them.

BEHIND THE SCENES

The landing page already refers to files that will be created in later lessons.

This is intentional.

The browser reads:

```
<link rel="stylesheet" href="styles.css">
```

and looks for a file named styles.css inside the same folder.

It also reads:

```
<script src="script.js"></script>
```

and looks for script.js inside the same folder.

The page remains usable even though those files are not present yet, but it will not have its final appearance or interactive navigation until they are created.

THINK LIKE A SOFTWARE DESIGNER

A public website should help visitors understand the software before asking them to create an account.

The sections should answer the visitor's questions in a sensible order.

First, explain the software.

Next, explain the business problem.

Then show the features and how the system works.

Finally, ask the visitor to register.

This creates a clear journey instead of presenting disconnected information.

CODE-READING QUESTION

Open index.html. Find the part of the page added for building the landing page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create the main HTML file for a business application
- structure a professional landing page
- connect future CSS and JavaScript files correctly
- prepare navigation for future authentication pages
- build a hamburger button with three span elements
- save an HTML file correctly using Notepad

In the next lesson, you will create styles.css and transform the plain landing page into a polished, responsive business website.

CHAPTER 1**BUILDING THE PUBLIC WEBSITE****LESSON 3**

STYLING THE LANDING PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the stylesheet for your Sales Tracker.

The HTML structure is already complete.

You will now transform it into a professional business website suitable for a real software product.

The responsive navigation will be completed in the next lesson when you create the JavaScript file.

WHY THIS MATTERS

Without CSS, a webpage contains only structure.

CSS controls:

- Colours
- Typography
- Layout
- Spacing
- Buttons
- Cards
- Responsive behaviour

A professional design helps visitors understand the software more quickly and creates confidence in the product.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ✓ index.html opens correctly.
- ✓ Every landing page section appears.
- ✓ The browser does not display HTML code as plain text.

Open index.html in Notepad.

Copy the complete code.

You will paste it into the Build Prompt so ChatGPT can style the exact HTML structure you have already created.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete HTML file named:

index.html

for my Sales Tracker.

Create one complete CSS file named:

styles.css

Return only the complete styles.css file.

Do not return HTML.

Do not return JavaScript.

Do not return explanations.

Do not return snippets.

Everything already built must continue working.

The stylesheet must match the exact HTML structure that I will paste below.

The HTML already contains this stylesheet link:

```
<link rel="stylesheet" href="styles.css">
```

GENERAL DESIGN

Create a clean, modern business software design.

Use a professional colour palette suitable for business applications.

Create CSS variables inside:

:root

Include variables for:

- Primary colour
- Secondary colour
- Accent colour
- Background colour
- Card colour
- Text colour
- Border colour
- Shadow
- Border radius
- Content width

Use:

box-sizing: border-box;

for every element.

Remove unnecessary browser margins and padding.

Use a professional system font stack.

Do not import fonts.

GENERAL LAYOUT

Create a centred content container.

Give each major section comfortable spacing.

Prevent content from touching the edges of the screen.

Create balanced spacing between headings, paragraphs, cards and buttons.

HEADER

Create a professional header.

Display:

Sale Records

clearly as the brand.

Arrange navigation horizontally on larger screens.

Style:

Login

as a secondary button.

Style:

Register

as the primary action.

Include hover and keyboard focus styles.

HAMBURGER BUTTON

Style the existing hamburger button.

The button already contains exactly three span elements.

Do not create additional lines using CSS pseudo-elements.

Hide the hamburger button on larger screens.

Display it on smaller screens.

Prepare the navigation menu so it becomes visible whenever JavaScript adds an:

active

class.

Do not write JavaScript.

HERO SECTION

Create an attractive hero layout.

Display the content and dashboard preview side by side on wider screens.

Stack them on smaller screens.

Make:

Get Started

the strongest visual button.

Style:

See How It Works

as a secondary button.

BUSINESS PROBLEM SECTION

Give this section a subtle visual difference from the hero section.

Use generous spacing.

Make the problem easy to read.

FEATURES SECTION

Display the six feature cards in a responsive grid.

Each card should include:

- Rounded corners
- Professional shadow
- Comfortable spacing
- Subtle hover effect

HOW IT WORKS SECTION

Display the four steps using attractive cards.

Clearly distinguish the step number from the description.

DASHBOARD PREVIEW

Style the HTML dashboard preview so it resembles a real business dashboard.

Style:

- Summary cards
- Search area
- Sale cards
- View Sale buttons

Do not depend on images.

FINAL CALL-TO-ACTION

Create a strong closing section.

Highlight the:

Create Your Account

button.

FOOTER

Create a professional footer.

Arrange footer content neatly.

Style footer navigation links.

RESPONSIVE DESIGN

Create responsive layouts for:

Desktop

Tablet

Mobile

On smaller screens:

Hide the desktop navigation.

Display the hamburger button.

Prepare the navigation to become visible when JavaScript adds the:

active

class.

Stack multi-column layouts vertically.

Prevent horizontal scrolling.

Keep buttons large enough to tap comfortably.

ACCESSIBILITY

Create visible keyboard focus styles.

Maintain good colour contrast.

Respect reduced-motion preferences where appropriate.

TECHNICAL REQUIREMENTS

Match the exact class names used in the supplied HTML.

Do not invent a different HTML structure.

Do not use CSS frameworks.

Do not use external fonts.

Return one complete:

styles.css

file.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

styles.css

The file should:

- ☒ Match the HTML.
- ☒ Style every section.
- ☒ Support desktop, tablet and mobile layouts.
- ☒ Prepare the navigation for JavaScript.
- ☒ Include no HTML.
- ☒ Include no JavaScript.

SAVE YOUR FILES

Copy the complete CSS returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Sales Tracker folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

styles.css

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

TEST YOUR WORK

Open:

index.html

If it is already open in your browser, refresh the page.

The landing page should now have a professional appearance.

Confirm that:

- ☒ Header is styled.
- ☒ Navigation is styled.
- ☒ Hero section looks professional.
- ☒ Dashboard preview resembles business software.
- ☒ Feature cards display correctly.
- ☒ Buttons look professional.
- ☒ Footer is styled.

Now reduce the browser width.

Confirm that:

- ☒ Layout adjusts correctly.

☒ Hamburger button appears.

☒ Desktop navigation disappears.

The hamburger button does not need to work yet.

That functionality will be added in the next lesson.

CHECKPOINT

Before moving on, confirm that:

☒ styles.css exists.

☒ The landing page is styled.

☒ Responsive layouts work.

☒ Hamburger button appears on smaller screens.

☒ Buttons display correctly.

☒ Dashboard preview looks professional.

☒ Footer is styled.

COMMON BEGINNER MISTAKES

A common mistake is saving the file as:

styles.css.txt

Always choose:

Save as type:

All Files

Another common mistake is changing class names inside the CSS.

The stylesheet must match the HTML structure exactly.

If ChatGPT returns styles that do not match your HTML, provide your complete index.html again and ask for a complete updated styles.css file.

Do not ask for snippets.

BEHIND THE SCENES

When your browser opens index.html, it immediately reads:

```
<link rel="stylesheet" href="styles.css">
```

It then loads every CSS rule inside styles.css and applies those rules to matching HTML elements.

This is why HTML and CSS must always stay in sync.

If an HTML class changes but the CSS is not updated, the styling may stop working.

THINK LIKE A SOFTWARE DESIGNER

Business software should communicate trust.

Professional colours, balanced spacing, consistent buttons, and organised layouts help users feel confident before they even create an account.

Good design is not decoration.

Good design makes software easier to understand and easier to use.

CODE-READING QUESTION

Open styles.css. Find one style rule added for styling the landing page. Which selector does the rule use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a professional stylesheet
- style a business landing page
- prepare responsive layouts

- style dashboard preview cards
- prepare the navigation for JavaScript
- keep HTML and CSS working together

In the next lesson, you will create `script.js` and make the responsive navigation fully functional.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 4

ADDING THE RESPONSIVE NAVIGATION

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the JavaScript that makes the responsive navigation work.

Your landing page already contains:

- HTML structure
- Professional styling
- A three-line hamburger button

The only missing piece is the behaviour.

When the browser window becomes narrow, visitors should be able to open and close the navigation menu using the hamburger button.

WHY THIS MATTERS

Many people will visit your website using a mobile phone or tablet.

A navigation menu that works well on a large computer screen may become difficult to use on a smaller device.

Responsive navigation solves this problem by allowing users to open and close the menu whenever they need it.

This creates a cleaner layout and improves the overall user experience.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that your project folder contains:

index.html

styles.css

Open index.html in your browser.

Reduce the browser width.

You should see:

☒ The hamburger button.

☒ The normal navigation hidden.

The hamburger button will not do anything yet.

That is expected.

Open index.html in Notepad.

Copy the complete code.

Also open styles.css in Notepad and copy the complete code.

You will paste both files into the Build Prompt so ChatGPT can use the exact IDs and class names already in your project.

BUILD PROMPT

PROMPT STARTS HERE

I already have these files in my project:

index.html

styles.css

Everything already built must continue working.

Create one complete JavaScript file named:

script.js

Return only the complete script.js file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

- Use plain Vanilla JavaScript.
- Do not use any framework.
- Do not use external libraries.
- Do not use Supabase.
- Do not add authentication code.
- Do not add sales management code.

RESPONSIVE NAVIGATION

- Use the exact HTML structure that I provide.
- Find:
 - The hamburger button.
 - The navigation menu.
- Use the exact IDs or class names already present.
- When the hamburger button is clicked:
 - Open the navigation menu.
 - Add the existing active class expected by styles.css.
- When the hamburger button is clicked again:
 - Close the navigation menu.
 - Remove the active class.

ACCESSIBILITY

- Update:
 - aria-expanded
 - correctly.
- When the menu opens:
 - aria-expanded="true"
- When the menu closes:
 - aria-expanded="false"
- Do not remove any accessibility attributes already present.

CLOSING THE MENU

When the visitor clicks any navigation link inside the mobile menu:

- Close the menu.

When the visitor presses the Escape key:

- Close the menu.

When the visitor clicks anywhere outside the navigation:

- Close the menu.

When the browser window becomes wider than the mobile layout:

- Close the mobile menu automatically.

SMOOTH NAVIGATION

Allow links pointing to sections inside:

`index.html`

to move smoothly.

Do not interfere with links pointing to:

`login.html`

`register.html`

Those links must continue opening normally.

CODE QUALITY

Wait until the page has loaded before selecting HTML elements.

Check that required elements exist before attaching event listeners.

Use clear variable names.

Avoid duplicate event listeners.

Do not include unfinished comments.

Return one complete `script.js` file.

IMPORTANT

Everything already built must continue working.

Use the exact IDs and class names already present inside my HTML.

Do not invent different selectors.

The active class must match the class already prepared inside styles.css.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

Here is my complete styles.css file:

[PASTE YOUR COMPLETE STYLES.CSS HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

script.js

The file should:

- ☒ Open the mobile navigation.
- ☒ Close the mobile navigation.
- ☒ Update aria-expanded.
- ☒ Close after selecting a navigation link.
- ☒ Close after pressing Escape.
- ☒ Close after clicking outside the navigation.
- ☒ Close automatically when returning to desktop width.
- ☒ Leave Login and Register links working normally.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Sales Tracker folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

script.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

Make sure the file is not named:

script.js.txt

TEST YOUR WORK

Open:

index.html

If it is already open, refresh the browser.

Reduce the browser width until the hamburger button appears.

Click the hamburger button.

The navigation menu should open.

Click the hamburger button again.

The navigation menu should close.

Open the menu.

Click:

Features

The menu should close automatically.

The browser should move to the Features section.

Open the menu again.

Press the Escape key.

The menu should close.

Open the menu again.

Click outside the navigation.

The menu should close.

Finally, widen the browser window.

The mobile menu should close automatically and the normal desktop navigation should appear again.

Click:

Login

The browser may display a page-not-found message.

This is expected because login.html has not yet been created.

Click:

Register

The browser may also display a page-not-found message.

This is expected because register.html will be created later in Workbook 06.

CHECKPOINT

Before moving on, confirm that:

- ☒ script.js exists.
- ☒ The hamburger button opens the menu.
- ☒ The hamburger button closes the menu.
- ☒ Navigation links close the menu.
- ☒ Escape closes the menu.

- ✓ Clicking outside closes the menu.
- ✓ Returning to desktop width closes the mobile menu.
- ✓ Login still points to login.html.
- ✓ Register still points to register.html.

COMMON BEGINNER MISTAKES

A common mistake is changing the class name used by JavaScript without updating the CSS.

The JavaScript and CSS must both use the same active class.

Another common mistake is searching for HTML elements using IDs or class names that do not exist.

Always use the exact HTML already present in your project.

If the menu does not open, provide ChatGPT with both your complete index.html and styles.css files and ask it to regenerate the complete script.js file.

Do not ask for only a small correction.

BEHIND THE SCENES

JavaScript does not create a second navigation menu.

Instead, it changes the state of the menu that already exists inside index.html.

When the active class is added, CSS displays the menu.

When the active class is removed, CSS hides it again.

The browser simply switches between those two states.

THINK LIKE A SOFTWARE DESIGNER

Responsive navigation should feel natural.

Users should never wonder how to close the menu or whether the application has stopped responding.

Good software provides predictable behaviour in every situation.

Small details like automatically closing the menu after a navigation link is selected make an application feel much more polished.

CODE-READING QUESTION

Open script.js. Find one function that supports adding the responsive navigation. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a JavaScript file
- control responsive navigation
- update accessibility attributes
- respond to user interactions
- coordinate HTML, CSS and JavaScript
- improve the mobile experience

In the next lesson, you will review the complete public website, improve its overall quality, and prepare it for the Supabase connection in Chapter 2.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 5

COMPLETING THE PUBLIC WEBSITE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will review and improve the complete public website for your Sales Tracker.

You have already built:

- `index.html`
- `styles.css`
- `script.js`

Your website is functional, but like most software, it can still be improved.

Rather than adding new features, you will refine what already exists while preserving every working part of the project.

WHY THIS MATTERS

Professional software is rarely perfect after the first version.

Developers continuously improve layouts, wording, accessibility, responsiveness, spacing, and usability.

These improvements make the software easier to understand and more pleasant to use without changing its purpose.

Learning how to improve existing software safely is one of the most valuable software development skills.

BEFORE YOU CONTINUE

Confirm that your Sales Tracker folder contains:

`index.html`

styles.css

script.js

Open index.html in your browser.

Confirm that:

☒ The landing page opens.

☒ The page is fully styled.

☒ The responsive navigation works.

☒ The Login link points to:

login.html

☒ The Register link points to:

register.html

Do not continue until everything above works correctly.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete public website for my Sales Tracker.

Everything already built must continue working.

Do not remove any existing functionality.

Return complete updated versions of every affected file.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

index.html

styles.css

script.js

only if improvements are required.

GENERAL GOAL

Review the complete public website and improve its overall quality while preserving every existing feature.

INDEX.HTML

Keep every existing section.

Do not remove:

- Header
- Navigation
- Hero
- Business Problem
- Features
- How It Works
- Dashboard Preview
- Final Call-to-Action
- Footer

Improve where appropriate:

- Heading wording
- Paragraph wording
- Accessibility labels
- Semantic HTML

Do not rename IDs unless absolutely necessary.

Keep:

```
<link rel="stylesheet" href="styles.css">
```

Keep:

```
<script src="script.js"></script>
```

Continue pointing:

Login

to:

login.html

Continue pointing:

Register

to:

register.html

STYLES.CSS

Keep every existing style.

Improve where appropriate:

- Consistent spacing
- Typography
- Button styling
- Card alignment
- Section spacing
- Mobile layout
- Dashboard preview appearance
- Footer layout

Improve keyboard focus styles where appropriate.

Maintain good colour contrast.

Do not redesign the page completely.

Refine the existing design.

SCRIPT.JS

Keep every existing feature working.

Review the JavaScript for:

- Duplicate event listeners
- Repeated logic
- Unnecessary code
- Accessibility improvements

Continue supporting:

- Responsive navigation
- Hamburger menu
- Navigation closing
- Escape key

- Outside click
 - Desktop resize handling
- Do not add authentication code.
- Do not add Supabase code.
- Do not add sales management code.

QUALITY REVIEW

Review the complete website and ensure:

- Navigation feels smooth.
- Buttons are consistent.
- Messages and wording are professional.
- Layout spacing is balanced.
- Mobile experience is clean.
- Keyboard navigation still works.

IMPORTANT

- Everything already built must continue working.
- Do not remove features.
- Do not rename files.
- Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

- ChatGPT should return complete updated versions of any files that require improvement.
- Every returned file should be complete from beginning to end.
- Nothing that already works should stop working.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

If ChatGPT returns an updated version of any file:

Open that file in Notepad.

Delete the existing contents.

Paste the complete updated code.

Click File.

Click Save.

Repeat this process for every updated file.

Do not create duplicate files.

Always replace the complete contents of the existing file.

Your project folder should still contain only:

index.html

styles.css

script.js

TEST YOUR WORK

Refresh:

index.html

Review the complete landing page.

Confirm that:

- ☒ The header still appears correctly.
- ☒ Navigation still works.
- ☒ The hamburger menu still works.
- ☒ Every section remains present.
- ☒ Buttons remain consistent.
- ☒ The dashboard preview still looks professional.
- ☒ The footer still appears correctly.

Now reduce the browser width.

Confirm that:

- ☒ The layout adjusts correctly.
- ☒ The hamburger menu still works.
- ☒ No content runs outside the page.
- ☒ No unnecessary horizontal scrolling appears.

Finally, click:

Home

Features

How It Works

Login

Register

Confirm that:

- ☒ Internal navigation still works.
- ☒ Login still points to login.html.
- ☒ Register still points to register.html.

CHECKPOINT

Before moving on, confirm that:

- ☒ The landing page looks professional.

- ✓ Every section remains present.
- ✓ Navigation still works.
- ✓ Responsive behaviour still works.
- ✓ Login and Register links remain correct.
- ✓ No existing functionality has been lost.

COMMON BEGINNER MISTAKES

Some beginners ask ChatGPT to improve only a small section of a file.

That often causes important code to disappear.

Always request the complete updated file.

Another common mistake is accepting improvements without testing the complete website afterwards.

Even small changes can accidentally affect responsive layouts or navigation.

Refresh the browser and test the whole page after every update.

BEHIND THE SCENES

As software projects become larger, developers rarely rebuild them from scratch.

Instead, they improve existing files while preserving the features that already work.

Throughout this workbook, you will continue following this same approach.

Each Build Prompt will protect everything already built while adding or refining only what is necessary.

THINK LIKE A SOFTWARE DESIGNER

Professional software evolves through continuous improvement.

Instead of asking,

"What can I build next?"

experienced developers often ask,

"What can I make better?"

Learning to improve existing software without breaking it is one of the habits that separates experienced developers from beginners.

CODE-READING QUESTION

Open index.html. Find the part of the page added for completing the public website. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an existing project
- improve a public website safely
- preserve existing functionality
- refine layouts and wording
- test a website after improvements

CHAPTER SUMMARY

Congratulations.

You have completed the public website for your Sales Tracker.

Your application now has a professional landing page that explains the software clearly and guides visitors towards creating an account.

You also built a responsive navigation system that works across different screen sizes.

Most importantly, you have followed a clear professional workflow.

You built the application one reliable step at a time, testing each stage before moving forward.

CHAPTER MILESTONE

By the end of Chapter 1, you have successfully built:

- ✓ Professional landing page
- ✓ Responsive navigation
- ✓ Three-line hamburger menu
- ✓ Hero section
- ✓ Business problem section
- ✓ Features section
- ✓ How It Works section
- ✓ Dashboard preview
- ✓ Final call-to-action
- ✓ Footer
- ✓ Login link
- ✓ Register link
- ✓ Responsive layout
- ✓ HTML, CSS and JavaScript working together

Your project folder should now contain:

index.html

styles.css

script.js

TRANSITION TO CHAPTER 2

Your public website is now complete.

Visitors can learn about your software and navigate through the landing page.

The next step is to connect your application to Supabase.

In Chapter 2, you will create your Supabase project, build a shared connection file, test the connection, and prepare the application for the complete authentication system that follows.

Once that foundation is complete, you will begin building secure user accounts and protected sales management features.

CHAPTER 2

CONNECTING TO SUPABASE

CHAPTER INTRODUCTION

Your public website is now complete.

Visitors can learn about your Sales Tracker, understand how it works, and choose to register for an account.

The next step is to give your application a secure online backend.

That backend is Supabase.

Supabase will be responsible for:

- User authentication
- Sale data storage
- Database security
- Protecting every user's sales

During this chapter, you will connect your website to your own Supabase project.

Although visitors will not register until Chapter 3, everything required to support registration and sales management will already be in place.

Think of this chapter as laying the foundation before constructing the building.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Create a Supabase project
- Locate the Project URL
- Locate the Publishable Key
- Create a shared connection file

- Build a connection test page
- Verify that your application communicates successfully with Supabase

By the end of the chapter, your project will be fully prepared for authentication and sales management.

LESSON 1

CREATING YOUR SUPABASE PROJECT

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a new Supabase project specifically for your Sales Tracker.

Each workbook should have its own Supabase project.

Keeping projects separate makes them easier to manage and prevents one application from accidentally affecting another.

WHY THIS MATTERS

Your Sales Tracker needs a secure online backend.

Without one, users cannot:

- Create accounts
- Sign in
- Store sales
- Retrieve sales later

Supabase provides all of these services.

BEFORE YOU CONTINUE

Make sure you have:

- ☒ A working internet connection.
- ☒ A web browser.
- ☒ Your Sales Tracker folder.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create your Supabase project.

Follow these steps.

1.

Open your web browser.

2.

Visit:

<https://supabase.com>

3.

Create a free account if you do not already have one.

4.

Sign in.

5.

Click:

New Project

6.

Choose your organisation.

7.

Enter a project name.

For example:

Sales Tracker

8.

Create a strong database password.

Store it somewhere safe.

Do not share it with anyone.

Your application will not use this password directly.

9.

Choose the region closest to you.

10.

Click:

Create New Project

Wait while Supabase prepares the project.

This may take several minutes.

Do not continue until the project is ready.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your web browser.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

After the project has been created successfully:

Open the project dashboard.

Confirm that:

- ☒ The project opens.
- ☒ The project status shows that it is ready.

CHECKPOINT

Before moving on, confirm that:

- ☒ Your Supabase account exists.
- ☒ Your new project has been created.
- ☒ The project dashboard opens successfully.
- ☒ The project is ready.

COMMON BEGINNER MISTAKES

Some students close the browser before Supabase finishes creating the project.

Always wait until the project is fully ready.

Another common mistake is forgetting the database password.

Store it somewhere safe before leaving this page.

Remember that your website will not use this password.

Later lessons will use the Publishable Key instead.

BEHIND THE SCENES

When Supabase creates your project, it automatically prepares:

- A PostgreSQL database
- User authentication
- Storage services
- Security settings
- APIs

These services work together to support the software you are building.

THINK LIKE A SOFTWARE DESIGNER

Professional developers normally create a separate backend for each application.

Keeping projects separate reduces confusion, improves security, and makes future maintenance much easier.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a Supabase project
- prepare an online backend

- understand why every software project should have its own database

In the next lesson, you will locate the Project URL and Publishable Key that allow your Sales Tracker to communicate securely with Supabase.

CHAPTER 2**CONNECTING TO SUPABASE****LESSON 2**

UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will locate the two pieces of information your Sales Tracker needs to communicate with Supabase.

These are:

- The Project URL
- The Publishable Key

In the next lesson, you will place these values inside a reusable connection file that every page in your application will use.

WHY THIS MATTERS

Your website needs to know where your Supabase project is located before it can communicate with it.

The Project URL identifies your Supabase project.

The Publishable Key allows your website to communicate with that project securely.

These two values will be used throughout this workbook whenever your application:

- Registers a user
- Signs in a user
- Loads sales
- Saves sales
- Updates sales

- Deletes sales

BEFORE YOU CONTINUE

Confirm that:

- ☒ Your Supabase project has finished creating.
- ☒ You are signed in to Supabase.
- ☒ Your project dashboard is open.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, locate your Project URL and Publishable Key.

Follow these steps carefully.

1.

Open your Supabase project.

2.

From the left-hand menu, click:

Project Settings

3.

Click:

Data API

Locate:

Project URL

It will look similar to:

<https://your-project-id.supabase.co>

Copy the complete value.

Do not change it.

Next, locate:

Publishable Key

Copy the complete key.

Store both values somewhere safe.

You will use them in the next lesson.

IMPORTANT

Your Project URL must end with:

.supabase.co

Do not add:

/rest/v1/

The correct format is:

`https://your-project-id.supabase.co`

IMPORTANT

Use only the:

Publishable Key

Never use:

- Service Role Key
- Secret Key
- Database password

Only the Publishable Key belongs inside your project files.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your Supabase dashboard.

SAVE YOUR FILES

No project files are created during this lesson.

Keep your Project URL and Publishable Key ready for the next lesson.

TEST YOUR WORK

Confirm that you have copied:

- ☒ Your Project URL
- ☒ Your Publishable Key

Check the Project URL carefully.

It should end with:

.supabase.co

There should be nothing after that.

CHECKPOINT

Before moving on, confirm that:

- ☒ You have located your Project URL.
- ☒ You have located your Publishable Key.
- ☒ You copied the Publishable Key.
- ☒ You did not copy a secret key.
- ☒ Your Project URL does not contain:

/rest/v1/

COMMON BEGINNER MISTAKES

One common mistake is copying the wrong key.

Supabase displays several different keys.

Only the Publishable Key should be used in this workbook.

Never use the Service Role Key.

Never use a Secret Key.

Those keys are intended for secure server environments and should never appear inside files that visitors can download through their web browser.

Another common mistake is copying the REST API address instead of the Project URL.

Always use the base Project URL ending with:

.supabase.co

BEHIND THE SCENES

Whenever your application communicates with Supabase, it sends requests to your Project URL.

Those requests include your Publishable Key.

Supabase uses this information to identify your project before processing the request.

Although the Publishable Key is visible inside your website, it does not give visitors unrestricted access to your data.

Authentication and Row Level Security will provide that protection later in this workbook.

THINK LIKE A SOFTWARE DESIGNER

Professional software separates public information from private information.

The Project URL and Publishable Key are intended to be used by browser-based applications.

Sensitive credentials remain on secure servers and are never exposed to visitors.

Understanding this difference is one of the foundations of building secure software.

WHAT YOU LEARNED

In this lesson you learned how to:

- locate the Project URL
- locate the Publishable Key
- understand the purpose of both values
- avoid using secret keys
- prepare your project for the shared Supabase connection

In the next lesson, you will create a reusable Supabase connection file that every page in your Sales Tracker will use.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 3

CREATING YOUR SUPABASE CONNECTION FILE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a reusable JavaScript file that connects your Sales Tracker to your Supabase project.

Rather than writing the connection code on every page, you will create one shared file named:

`supabase-config.js`

Every page that needs to communicate with Supabase will use this file.

This keeps your project organised and makes future updates much easier.

WHY THIS MATTERS

As software grows, several pages often need to communicate with the same database.

Instead of copying the connection code into every JavaScript file, professional developers create one shared connection file.

Whenever another page needs Supabase, it simply loads this file.

This approach reduces mistakes, keeps the project easier to maintain, and ensures every page uses exactly the same connection.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ Your Supabase project has been created.

✓ You have copied your Project URL.

✓ You have copied your Publishable Key.

Your Sales Tracker folder should currently contain:

index.html

styles.css

script.js

You are about to add a fourth file.

BUILD PROMPT

PROMPT STARTS HERE

Create one complete JavaScript file named:

supabase-config.js

Return only the complete file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

Create a reusable Supabase connection file.

Use the official Supabase browser client.

Create the client using this format exactly:

```
const supabaseClient = window.supabase.createClient(  
  "https://your-project-id.supabase.co",  
  "YOUR_PUBLISHABLE_KEY"  
);
```

Replace the placeholder values with my actual values.

Project URL:

PASTE YOUR PROJECT URL HERE

Publishable Key:

PASTE YOUR PUBLISHABLE KEY HERE

REQUIREMENTS

- Create only one Supabase client.
- Store it in a constant named:
`supabaseClient`
- Do not use:
`/rest/v1/`
inside the Project URL.
- Do not use:
Service Role Key
- Do not use:
Secret Key
- Do not use:
Database password
- Include a short comment at the top of the file explaining that this file creates the shared Supabase connection used throughout the application.
- Return one complete JavaScript file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`supabase-config.js`

The file should:

- ☒ Create one shared Supabase client.
- ☒ Use my Project URL.

☒ Use my Publishable Key.

☒ Create the client using:

```
const supabaseClient = window.supabase.createClient(...)
```

☒ Contain JavaScript only.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new empty document.

Click File.

Click Save As.

Open your Sales Tracker folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

supabase-config.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

Make sure the file is not named:

supabase-config.js.txt

TEST YOUR WORK

Open your Sales Tracker folder.

Confirm that:

supabase-config.js

appears alongside the other project files.

Open the file in Notepad.

Confirm that:

✓ The Project URL ends with:

.supabase.co

✓ The Project URL does not contain:

/rest/v1/

✓ The Publishable Key appears.

✓ The Supabase client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

At this stage, the connection file is not yet being used by any webpage.

That is expected.

The connection will be tested in the next lesson.

CHECKPOINT

Before moving on, confirm that:

✓ supabase-config.js has been created.

✓ The filename is correct.

✓ The Project URL is correct.

✓ The Publishable Key is correct.

✓ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

One common mistake is copying the REST API address instead of the Project URL.

Always use the base URL ending with:

`.supabase.co`

Another common mistake is copying the Service Role Key instead of the Publishable Key.

Only the Publishable Key belongs inside this file.

Never place secret keys inside browser-based applications.

Some beginners also create another Supabase client later inside `register.js` or `dashboard.js`.

Do not do that.

Throughout this workbook, every page will use the single shared client created inside:

`supabase-config.js`

BEHIND THE SCENES

The line:

```
const supabaseClient = window.supabase.createClient(...)
```

creates one reusable connection between your application and your Supabase project.

Later in this workbook, every page that needs Supabase will simply use:

`supabaseClient`

instead of creating another connection.

This keeps the application consistent and greatly reduces the chance of connection errors.

THINK LIKE A SOFTWARE DESIGNER

Whenever several parts of an application need the same functionality, it is usually better to create that functionality once and reuse it everywhere.

This principle keeps software simpler, easier to maintain, and less likely to contain conflicting code.

Creating a shared Supabase connection file is one example of that idea.

CODE-READING QUESTION

Open supabase-config.js. Which line creates the Supabase client, and which two saved values does that line use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a shared Supabase connection file
- create one reusable Supabase client
- use the correct Project URL
- use the correct Publishable Key
- avoid exposing secret keys
- prepare your application for future database communication

In the next lesson, you will build a connection test page that confirms your Sales Tracker can communicate successfully with your Supabase project.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 4

BUILDING A CONNECTION TEST PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a simple webpage that confirms your Sales Tracker can communicate successfully with your Supabase project.

This page exists only for testing.

It is not part of the finished application.

Instead, it allows you to verify that your shared Supabase connection is working correctly before you begin building authentication.

WHY THIS MATTERS

Professional developers test important parts of an application before building additional features.

Imagine spending hours creating registration and login pages only to discover that your website was never connected to Supabase correctly.

Testing the connection now helps you identify problems while the project is still simple.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

You should also have:

- ☒ Your Supabase project
- ☒ Your Project URL
- ☒ Your Publishable Key

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have these files:

index.html

styles.css

script.js

supabase-config.js

Everything already built must continue working.

Do not modify any existing files.

Create one new HTML file named:

test-connection.html

Return one complete HTML file only.

Do not return explanations.

Do not return snippets.

GENERAL REQUIREMENTS

Create a complete HTML5 document.

Include:

- Appropriate page title
- Viewport meta tag

Inside the head section, include simple CSS that creates a clean, centred layout suitable for a testing page.

BODY CONTENT

Display:

Heading

Supabase Connection Test

Below the heading, display a short paragraph explaining that the page is checking communication with the Supabase project.

Create a status box.

The status box should initially display:

Checking connection...

Give this element the ID:

connection-status

SCRIPT LOADING ORDER

Immediately before the closing body tag, load these scripts in this exact order.

First:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

Second:

```
<script src="supabase-config.js"></script>
```

Third:

Create an inline JavaScript block that:

- Checks whether:

supabaseClient

exists.

If it exists:

Change the status box to display:

✅ Connected successfully to Supabase.

Apply a success style.

If it does not exist:

Display:

❌ Unable to connect to Supabase.

Apply an error style.

IMPORTANT

Use the existing:

supabase-config.js

Do not create another Supabase client.

Do not hard-code the Project URL.

Do not hard-code the Publishable Key.

Do not use authentication.

Do not create database tables.

Do not create users.

Return one complete HTML file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

test-connection.html

The page should:

- ☒ Load the Supabase browser library.
- ☒ Load supabase-config.js.
- ☒ Load scripts in the correct order.
- ☒ Check whether supabaseClient exists.
- ☒ Display a clear success or error message.

SAVE YOUR FILES

Copy the complete HTML returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Sales Tracker folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

test-connection.html

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TEST YOUR WORK

Open:

test-connection.html

The page should open in your browser.

After a moment, the message should change from:

Checking connection...

to:

✅ Connected successfully to Supabase.

If instead you see:

❌ Unable to connect to Supabase.

Do not continue.

Return to the previous lessons and carefully check:

- ✓ Project URL
- ✓ Publishable Key
- ✓ supabase-config.js
- ✓ Script loading order

CHECKPOINT

Before moving on, confirm that:

- ✓ test-connection.html opens.
- ✓ The page loads correctly.
- ✓ The Supabase browser library loads first.
- ✓ supabase-config.js loads second.
- ✓ The page displays:

Connected successfully to Supabase.

COMMON BEGINNER MISTAKES

The most common mistake is loading the JavaScript files in the wrong order.

The correct order is always:

1. Supabase browser library
2. supabase-config.js
3. Page-specific JavaScript

Another common mistake is creating another Supabase client inside the page.

Do not do this.

Your application should use only the shared client created inside:

supabase-config.js

BEHIND THE SCENES

This page does not create a new connection.

Instead, it loads the shared connection file.

If that file creates:

`supabaseClient`

successfully, the page simply confirms that the shared connection is available.

Every protected page you build later will use this same approach.

THINK LIKE A SOFTWARE DESIGNER

Professional software is built on reliable foundations.

Testing one important component before moving to the next makes larger projects much easier to debug.

Finding problems early almost always saves time later.

CODE-READING QUESTION

Open `test-connection.html`. Find the part of the page added for building a connection test page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a dedicated connection test page
- load the Supabase browser library
- reuse a shared connection file
- verify that your application can communicate with Supabase
- identify common connection problems

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 5

FINAL CONNECTION REVIEW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

This lesson is your final review before building the authentication system.

You are not creating any new files.

Instead, you will carefully inspect your project and confirm that every connection to Supabase has been configured correctly.

WHY THIS MATTERS

The authentication system depends completely on your Supabase connection.

If the connection is incorrect, registration, login, sales management and every future database feature will fail.

Confirming your work now helps prevent much larger problems later.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, review your project carefully.

Complete the following checks.

Open:

supabase-config.js

Confirm that:

☒ The Project URL ends with:

.supabase.co

☒ The Project URL does not contain:

/rest/v1/

☒ The Publishable Key is being used.

☒ The shared client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

Next, open:

test-connection.html

Confirm that the scripts load in this exact order:

1.

Supabase browser library

2.

supabase-config.js

3.

The page's own JavaScript

Finally, open:

test-connection.html

Confirm that the page displays:

☒ Connected successfully to Supabase.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by reviewing your project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this checklist.

Landing Page

☒ index.html opens correctly.

☒ Navigation works.

☒ Responsive menu works.

Supabase

☒ Project created.

☒ Project URL correct.

☒ Publishable Key correct.

☒ Shared connection file created.

Connection Test

☒ test-connection.html opens.

☒ Connected successfully to Supabase appears.

Project Folder

☒ index.html

☒ styles.css

☒ script.js

✓ supabase-config.js

✓ test-connection.html

CHECKPOINT

Before moving to Chapter 3, confirm that:

✓ Every required file exists.

✓ The landing page still works.

✓ The connection test succeeds.

✓ The Project URL is correct.

✓ The Publishable Key is correct.

✓ The script loading order is correct.

✓ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

Some students continue building after the connection test has failed.

Do not do this.

Authentication depends on the connection working correctly.

Solve connection problems now before moving to the next chapter.

BEHIND THE SCENES

At this point, your Sales Tracker knows how to communicate with Supabase.

What it cannot do yet is identify users.

That is the purpose of the next chapter.

Authentication allows the application to recognise who is currently signed in so every sale can be linked to the correct person.

THINK LIKE A SOFTWARE DESIGNER

Every large software project is built in layers.

Public website.

Backend connection.

Authentication.

Database.

Business features.

Building each layer carefully creates a much more reliable application than trying to build everything at once.

WHAT YOU LEARNED

In this lesson you learned how to:

- verify the shared Supabase connection
- review script loading order
- confirm that your application is ready for authentication
- understand why authentication comes before sales management

CHAPTER SUMMARY

Congratulations.

Your Sales Tracker is now successfully connected to Supabase.

You created your own Supabase project, located the Project URL and Publishable Key, built a shared connection file, and confirmed that your application can communicate successfully with your online backend.

More importantly, you established a reusable architecture that every future page in the application will use.

Whenever a page needs Supabase, it will always load:

1. The Supabase browser library.
2. `supabase-config.js`.
3. The page-specific JavaScript file.

Maintaining this structure throughout the project will keep your application organised and reduce connection-related problems.

CHAPTER MILESTONE

By the end of Chapter 2, you have successfully completed:

- ✓ Supabase project
- ✓ Project URL
- ✓ Publishable Key
- ✓ Shared connection file
- ✓ Connection test page
- ✓ Connection verification
- ✓ Correct script loading order
- ✓ Backend preparation for authentication

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TRANSITION TO CHAPTER 3

Your public website is complete.

Your application is connected to Supabase.

The next step is to build the complete authentication system.

In Chapter 3, you will build registration, login, the protected dashboard, and the protected sale details page as one coordinated system.

By creating every destination page together, every redirect will already have a valid destination, allowing the complete authentication workflow to function correctly from the very beginning.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

CHAPTER INTRODUCTION

Your Sales Tracker now has a professional public website and a working connection to Supabase.

The next step is to allow people to create their own accounts and access their own sales securely.

Authentication is one of the most important parts of any business application.

Without authentication, every visitor would have access to the same sale database.

That would make the software unsafe to use.

In this chapter, you will build the complete authentication system as one coordinated feature.

Instead of creating one page at a time, you will create every authentication page together.

This approach ensures that:

- Every redirect already has a destination.
- Every page is correctly linked.
- Every JavaScript file loads correctly.
- Every stylesheet is linked correctly.
- The complete user journey works from the very beginning.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- `register.html`
- `login.html`

- dashboard.html
- sale-details.html
- auth.css
- register.js
- login.js
- dashboard.js
- sale-details.js

You will also update:

- index.html

The completed authentication flow will work like this:

Visitor

↓

index.html

↓

Register

↓

register.html

↓

Email verification

↓

login.html

↓

Login

↓

dashboard.html

↓

Sale Details

↓

sale-details.html

↓

Logout

↓

login.html

Unauthenticated visitors who attempt to open:

dashboard.html

or

sale-details.html

will be redirected automatically to:

login.html

LESSON 1

UNDERSTANDING THE AUTHENTICATION FLOW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

Before creating the authentication system, it is important to understand how every page works together.

Authentication is not simply a login page.

It is a complete journey that begins when a visitor creates an account and continues until they safely sign out.

Every page depends on another page.

That is why the entire system will be built together.

WHY THIS MATTERS

Many beginners build authentication one page at a time.

The result is often broken redirects, missing files and incomplete user journeys.

Professional developers think about the complete workflow before writing any code.

Doing so reduces mistakes and produces a much more reliable application.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

- ✓ The landing page works.
- ✓ The Supabase connection test succeeds.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, study the authentication flow below.

A visitor arrives at:

`index.html`

The visitor selects:

Register

The visitor opens:

`register.html`

The visitor creates an account.

Supabase sends a verification email.

The visitor verifies the email address.

The visitor returns to:

`login.html`

The visitor signs in.

The application redirects to:

`dashboard.html`

The user manages sales.

Whenever the user opens a sale, the application opens:

`sale-details.html`

When the user signs out, the application redirects to:

`login.html`

If the visitor is not authenticated and tries to open:

`dashboard.html`

or

`sale-details.html`

the application immediately redirects them to:

```
login.html
```

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

Review the authentication flow.

Make sure you understand where every page sends the user.

Every destination page already has a purpose.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Registration comes before Login.
- ☒ Login comes before Dashboard.
- ☒ Sale Details is protected.
- ☒ Logout returns the user to login.html.
- ☒ Protected pages require authentication.

COMMON BEGINNER MISTAKES

One common mistake is thinking that a login page alone creates a complete authentication system.

A secure authentication system includes:

- Registration
- Email verification

- Login
- Protected pages
- Logout
- Automatic redirects

All of these parts work together.

BEHIND THE SCENES

Authentication allows the application to answer one important question:

Who is currently using the software?

Once that question has been answered, every sale can be connected to the correct user.

THINK LIKE A SOFTWARE DESIGNER

Always think about the complete journey.

Instead of asking:

"What happens on this page?"

ask:

"Where did the user come from?"

and

"Where should the user go next?"

That way, every page becomes part of one connected application.

WHAT YOU LEARNED

In this lesson you learned:

- the complete authentication workflow
- how every authentication page connects together
- why protected pages exist
- why redirects are planned before building begins

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

Estimated Time

75 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete authentication system for your Sales Tracker.

Rather than generating one page at a time, ChatGPT will generate every authentication page together.

This ensures that:

- Every destination page already exists.
- Every redirect already has somewhere to go.
- Every stylesheet is linked correctly.
- Every JavaScript file is loaded correctly.
- Everything already built continues working.

WHY THIS MATTERS

Registration, login, dashboard access and sale details are tightly connected.

Building them together reduces missing files, broken redirects and inconsistent behaviour.

This careful approach reduces missing files and broken redirects.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

☒ The connection test displays:

Connected successfully to Supabase.

Do not continue until everything above is working.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Sales Tracker.

Everything already built must continue working.

My project currently contains:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

The public website is complete.

The responsive navigation works.

The Supabase connection has been tested successfully.

Do not remove or break any existing functionality.

Return complete files only.

Do not return snippets.

Do not return explanations.

Return the following complete files:

1.

Updated index.html

2.

register.html

3.

login.html

4.

dashboard.html

5.

sale-details.html

6.

auth.css

7.

register.js

8.

login.js

9.

dashboard.js

10.

sale-details.js

GENERAL REQUIREMENTS

Everything already built must continue working.

Every HTML page must be complete.

Every HTML page must include proper HTML5 structure.

Every HTML page must include an appropriate page title.

Use semantic HTML.

Use modern, responsive layouts.

The logo on every page should display:

Sale Records

The logo should link to:

index.html

INDEX.HTML

Return a complete updated version.

Preserve:

- Hero
- Features
- Navigation
- Footer
- Responsive menu

Ensure Login points to:

login.html

Ensure Register points to:

register.html

REGISTER.HTML

Create a professional registration page.

Include:

- Logo
- Client Name
- Email Address
- Password
- Confirm Password
- Register button
- Link to login.html

Display friendly validation messages.

Disable the Register button while processing.

Display loading messages.

After successful registration:

Display a success message explaining that the user must verify their email before signing in.

Redirect the user to:

login.html

Do not automatically sign the user in.

LOGIN.HTML

Create a professional login page.

Include:

- Logo
- Email
- Password
- Login button
- Link to register.html

Display friendly validation and loading messages.

Disable the Login button while processing.

If login succeeds:

Redirect to:

dashboard.html

If the user is already authenticated:

Automatically redirect to:

dashboard.html

DASHBOARD.HTML

Create a professional dashboard page.

Include:

- Logo
- Welcome heading
- Logged-in user's email
- Four sale summary cards with temporary values:

Today's Sales Total

Today's Sale Count

This Month's Sales Total

All-Time Sales Total

Include a placeholder section where the sales management dashboard will be built later.

Include:

Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

SALE-DETAILS.HTML

Create a protected sale details page.

Include:

- Logo
- Sale details heading
- Placeholder sale information
- Back to Dashboard button
- Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

AUTH.CSS

Create one shared stylesheet for:

register.html

login.html

dashboard.html

sale-details.html

Use:

- Professional forms
- Responsive layout
- Business dashboard styling
- Summary cards
- Loading messages
- Success messages
- Error messages

REGISTER.JS

Load after:

1. Supabase CDN
2. supabase-config.js

Use:

```
const supabaseClient
```

already created inside:

```
supabase-config.js
```

Do not create another Supabase client.

Validate:

- Client Name
- Email
- Password
- Confirm Password

Passwords must match.

Create the account using Supabase Authentication.

When calling:

```
supabaseClient.auth.signUp
```

include an email confirmation destination.

Create it from the website that is currently open:

```
`${window.location.origin}/login.html`
```

Pass that address as:

```
emailRedirectTo
```

inside the sign-up options.

Do not hardcode a Netlify website name.

Using:

```
window.location.origin
```

allows the same complete files to work when the website address changes.

Display friendly success and error messages.

Disable the button while processing.

After successful registration:

Inform the user to open the verification email.

Redirect to:

login.html

Do not automatically sign the user in from the registration form.

LOGIN.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Authenticate users.

Display loading messages.

Display friendly errors.

Redirect successful logins to:

dashboard.html

DASHBOARD.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the user is authenticated.

Display the authenticated user's email.

Support Logout.

Redirect unauthenticated users to:

login.html

SALE-DETAILS.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the visitor is authenticated.

If not:

Redirect immediately to:

login.html

Support Logout.

Sales will be loaded later in this workbook.

For now, create the protected page and its authentication logic.

SCRIPT LOADING ORDER

Every Supabase-powered HTML page must load:

1.

Supabase browser library

2.

supabase-config.js

3.

Its own page-specific JavaScript

IMPORTANT

Return every requested file completely.

Do not omit any file.

Do not return partial updates.

Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return ten complete files.

Every HTML page should already contain the correct stylesheet and JavaScript links.

Every protected page should already verify authentication.

Every redirect should already point to an existing page.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2 (CONTINUED)

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

ChatGPT should return ten complete files.

Work through them one at a time.

If the file already exists, replace its complete contents.

If the file does not already exist, create it.

Do not copy only part of a file.

Always replace the complete contents.

Updated index.html

Open:

index.html

in Notepad.

Select all the existing code.

Cancel it.

Paste the complete updated code returned by ChatGPT.

Click:

File

Save

Close Notepad.

register.html

Open Notepad.

Create a new document.

Paste the complete code.

Click:

File

Save As

Open your Sales Tracker folder.

Choose:

Save as type:

All Files

Enter:

register.html

Click Save.

login.html

Repeat the same process.

Save as:

login.html

dashboard.html

Repeat the same process.

Save as:

dashboard.html

sale-details.html

Repeat the same process.

Save as:

sale-details.html

auth.css

Repeat the same process.

Save as:

auth.css

register.js

Repeat the same process.

Save as:

register.js

login.js

Repeat the same process.

Save as:

login.js

dashboard.js

Repeat the same process.

Save as:

dashboard.js

sale-details.js

Repeat the same process.

Save as:

sale-details.js

When you finish, your project folder should contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

sale-details.html

auth.css

register.js

login.js

dashboard.js

sale-details.js

Check every filename carefully.

Do not allow any file to become:

register.html.txt

login.js.txt

dashboard.html.txt

or any similar variation.

TEST YOUR WORK

Do not assume the authentication system works.

Authentication must be tested from a website with an HTTPS address.

Do not test this chapter by double-clicking the HTML files in your project folder.

The files still remain inside your project folder.

You will simply place a test copy of the complete folder online before testing.

BEFORE YOU START THE TESTS

1.

Save every updated file in Notepad.

2.

Deploy the complete project folder to Netlify.

If you already created a Netlify test website for this project, deploy the updated folder to the same website.

3.

Copy the website address supplied by Netlify.

It should begin with:

`https://`

For example:

`https://your-site-name.netlify.app`

Your own website address will be different.

4.

Open your Supabase project.

Go to:

Authentication

Then open:

URL Configuration

5.

Set the Site URL to your Netlify website address.

Do not add a file name to the Site URL.

6.

Add this complete address to Redirect URLs:

`https://your-site-name.netlify.app/login.html`

Replace the example website name with your real Netlify website name.

7.

Make sure the latest version of every file has been deployed.

8.

Open the project from its Netlify HTTPS address.

Keep using that address throughout every test below.

TEST 1

PUBLIC WEBSITE

Open the deployed index.html page.

Confirm that:

- ☒ The landing page opens correctly.
- ☒ The page is fully styled.
- ☒ The responsive navigation still works.
- ☒ Login opens the deployed login.html page.
- ☒ Register opens the deployed register.html page.

Look at the browser address bar.

The address must begin with:

https://

Nothing already built should have stopped working.

TEST 2

REGISTRATION PAGE

Open the deployed register.html page.

Attempt to submit the form without entering any information.

Friendly validation messages should appear.

Complete the form using an email address that has not already been used for this test.

Click:

Register

The Register button should become disabled while the request is being processed.

A loading message should appear.

After successful registration:

You should see a message asking you to verify your email.

TEST 3

EMAIL VERIFICATION

Open your email inbox.

Locate the verification email from Supabase.

Open the email.

Click the verification link.

Supabase should verify the email and return the browser to your deployed website.

The returned address should use your Netlify website name.

It should not begin with:

file://

Depending on the authentication response, you may briefly see login.html or the application may recognise the new session and open the dashboard.

Both results confirm that the browser returned to the correct website.

If the login page remains open, sign in using the email address and password you registered.

If the verification email does not arrive immediately:

Wait a few minutes.

Refresh your inbox.

Check your Spam or Junk folder.

If the verification link opens the wrong website:

Return to Supabase URL Configuration.

Check the Site URL and Redirect URLs carefully.

Correct them before creating another test account.

TEST 4

LOGIN

Open the deployed login.html page.

Enter an incorrect password.

A friendly error message should appear.

Now enter the correct password.

Click:

Login

The Login button should become disabled while signing in.

A loading message should appear.

After a successful login:

The browser should open the deployed dashboard.html page.

TEST 5

AUTHENTICATED SESSION

After signing in successfully:

Confirm that the protected dashboard opens.

Confirm that your email address appears.

Refresh the page.

The dashboard should remain open because the same website still has your authenticated session.

Confirm that the four sale summary cards appear.

Confirm that the Logout button appears.

While signed in, open the complete deployed sale-details.html address.

The protected sale details page should open.

Click:

Logout

The browser should return to the deployed login.html page.

While signed out, open the complete deployed dashboard.html address.

The application should immediately return you to the deployed login.html page.

While still signed out, open the complete deployed sale-details.html address.

The application should immediately return you to the deployed login.html page.

Sign in again.

While signed in, open the complete deployed login.html address.

The application should immediately return you to the deployed dashboard.html page.

CHECKPOINT

Before moving to the next lesson, confirm that:

- ☒ Registration works.
- ☒ Validation works.
- ☒ Loading messages appear.
- ☒ Email verification succeeds.
- ☒ Login works.
- ☒ Friendly error messages appear.
- ☒ Dashboard is protected.
- ☒ Sale Details page is protected.
- ☒ Logout works.
- ☒ Authenticated users cannot remain on login.html.
- ☒ Unauthenticated users cannot open dashboard.html.
- ☒ Unauthenticated users cannot open sale-details.html.

COMMON BEGINNER MISTAKES

One common mistake is forgetting to verify the email address before attempting to sign in.

If email verification is enabled, Supabase will not allow the account to be used until verification has been completed.

Another common mistake is loading the JavaScript files in the wrong order.

Every Supabase-powered page should load:

1.

Supabase browser library

2.

supabase-config.js

3.

The page-specific JavaScript

Changing this order can prevent the application from working correctly.

Some beginners also create another Supabase client inside:

register.js

login.js

dashboard.js

or

sale-details.js

Do not do this.

Every page should use the shared client created inside:

supabase-config.js

BEHIND THE SCENES

Your authentication system is now working as one connected application.

When a visitor creates an account, Supabase stores the authentication information securely.

After email verification, Supabase allows that account to sign in.

When login succeeds, Supabase creates a session.

Protected pages check whether that session exists before displaying their content.

If the session does not exist, the visitor is redirected immediately to the login page.

The Sale Details page already uses this protection even though sales have not yet been introduced.

Later, this same authentication system will protect every sale in the application.

THINK LIKE A SOFTWARE DESIGNER

Authentication is more than allowing someone to sign in.

It controls who can enter the application, which pages they can open, and what information they are allowed to see.

Every protected page should perform its own authentication check.

Never assume that because a user reached one protected page, every other page is automatically secure.

CODE-READING QUESTION

Open `register.js`. Find one function that supports building the complete authentication system. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete authentication system
- connect registration, login and protected pages
- protect multiple destinations
- redirect unauthenticated visitors
- redirect authenticated users appropriately
- preserve every feature already built

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 3

AUDITING THE AUTHENTICATION SYSTEM

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not build new features.

Instead, you will carefully review every part of the authentication system before beginning sales management.

Authentication is the foundation of the entire application.

Every sale created later will depend on the current user being identified correctly.

WHY THIS MATTERS

The next chapter introduces sales.

Every sale must belong to one authenticated user.

If authentication is unreliable, sale privacy cannot be guaranteed.

Professional developers verify security before adding business features.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

sale-details.html

auth.css

register.js

login.js

dashboard.js

sale-details.js

Confirm that you have successfully created at least one verified user account.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every authentication test again.

Do not continue until every test succeeds.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this final authentication check on the deployed HTTPS website.

Do not complete this audit from files opened with:

file://

SUPABASE URL SETTINGS

- ✓ The Site URL contains the Netlify website address.
- ✓ Redirect URLs contains the complete deployed login.html address.

REGISTRATION AND VERIFICATION

- ✓ Registration works from the deployed register.html page.
- ✓ A verification email arrives.
- ✓ The verification link returns to the deployed website.
- ✓ The returned address begins with https:// and uses the correct Netlify website name.

LOGIN AND SESSION

- ✓ Login works from the deployed login.html page.
- ✓ Refreshing a protected page keeps a signed-in user signed in.
- ✓ Logout works.

PROTECTION

- ✓ A signed-out visitor cannot remain on dashboard.html.
- ✓ A signed-in user can reach dashboard.html.
- ✓ A signed-in user who opens login.html returns to dashboard.html.

PROTECTED SALE PAGE

- ✓ A signed-in user can open sale-details.html.
- ✓ A signed-out visitor cannot remain on sale-details.html.

PAGES

- ✓ The signed-in user's email appears.
- ✓ The summary cards appear.

- ✓ The sale details placeholder appears.

If every check passes, the authentication system is ready for the next chapter.

CHECKPOINT

Before moving to Chapter 4, confirm that:

- ✓ Every authentication page works.
- ✓ Every redirect works.
- ✓ Dashboard protection works.
- ✓ Sale Details protection works.
- ✓ No authentication errors remain.

COMMON BEGINNER MISTAKES

Some students test only the successful login process.

Always test unsuccessful situations as well.

Protected pages should remain protected when no user is signed in.

Another common mistake is assuming one protected page automatically secures every other page.

Each protected page should verify authentication independently.

BEHIND THE SCENES

The authentication system now provides the identity that every future database operation will rely on.

When sales are introduced in Chapter 4, every new sale will automatically belong to the currently authenticated user.

Authentication and Row Level Security will work together to protect sale information.

THINK LIKE A SOFTWARE DESIGNER

Security should be confirmed before adding business functionality.

A reliable authentication system makes every future feature easier to build because the application always knows who the current user is.

WHAT YOU LEARNED

In this lesson you learned how to:

- audit an authentication system
- verify protected pages
- verify redirects
- prepare the application for secure sales management

CHAPTER SUMMARY

Congratulations.

Your Sales Tracker now includes a complete authentication system.

Visitors can register, verify their client email addresses, sign in, access protected pages, and sign out securely.

Every protected page already knows how to verify the current user's session before displaying information.

Most importantly, your application can now identify who is currently using the system.

That capability will allow every sale to belong to the correct user.

CHAPTER MILESTONE

By the end of Chapter 3, you have successfully built:

- ✓ User registration
- ✓ Email verification
- ✓ Secure login
- ✓ Protected dashboard
- ✓ Protected sale details page
- ✓ Logout
- ✓ Friendly validation
- ✓ Friendly success messages
- ✓ Friendly error messages
- ✓ Loading states
- ✓ Shared authentication styling
- ✓ Complete authentication workflow

TRANSITION TO CHAPTER 4

Your Sales Tracker can now identify every authenticated user.

The next step is to create the secure sale database.

In Chapter 4, you will design the sales table, create a richer business data model, enable Row Level Security, and prepare the application to store sale information safely.

This workbook introduces a complete database structure for managing sale details safely.

CHAPTER 4

BUILDING THE SALES DATABASE

CHAPTER INTRODUCTION

Your Sales Tracker can now recognise the person who is signed in.

The next step is to create a secure place for that person to store sales.

Each sale will record what was sold, when it was sold, how many units were sold and the amount charged for each unit. The database will calculate the total.

You will also protect every row with Row Level Security. This means one account cannot see or change another account's sales.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A sales data model
- The sales table
- Quantity, price and payment rules
- An automatically calculated total
- Row Level Security
- SELECT, INSERT, UPDATE and DELETE policies
- A two-account security test

LESSON 1

UNDERSTANDING THE SALES DATA MODEL

Estimated Time

20–30 Minutes

WHAT YOU ARE BUILDING

A clear plan for the information that one sale must contain.

WHY THIS MATTERS

A database becomes easier to build when you decide what each record means before writing SQL.

BEFORE YOU CONTINUE

- ☒ You can sign in on the deployed HTTPS website.
- ☒ Your Supabase project opens successfully.
- ☒ You have your Error Log nearby.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Help me plan one sales record before I create the database table.

Requirements:

- Use one row for one item sold in one sales entry.
- Include sale date, item name, category, quantity, unit price, total amount, payment method, customer name and notes.
- Explain which fields are required and which are optional.
- Explain in simple language why `user_id` is needed.
- Explain why `total_amount` should be calculated from quantity multiplied by `unit_price`.
- Do not write SQL yet.

Return the complete updated contents of:

- A plain-language data model plan

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A simple field-by-field plan.
- Clear required and optional field decisions.
- A short explanation of record ownership.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the plan in your project notes or workbook.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Read each field and describe what it stores.
 - ☒ Use a sample sale to check that every important detail has a place.
- Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ You understand what one row represents.
- ☒ You can explain how a sale total is calculated.

COMMON BEGINNER MISTAKES

- Treating one row as a whole shopping basket with unrelated items.
- Using text for quantity or prices.
- Forgetting the owner of the record.

BEHIND THE SCENES

A data model is a plan for stored information. Good field names make later prompts, forms and reports easier to understand.

THINK LIKE A SOFTWARE DESIGNER

If a business sells three units of the same item in one transaction, should that be one row with quantity 3 or three separate rows? Explain your choice.

WHAT YOU LEARNED

You learned:

- How to describe one sales record.
- Why data types and ownership matter.
- How quantity and unit price produce a total.

LESSON 2

CREATING THE SALES TABLE

Estimated Time

30–40 Minutes

WHAT YOU ARE BUILDING

The secure sales table and its basic database rules.

WHY THIS MATTERS

The table is the permanent home for every sale saved by the application.

BEFORE YOU CONTINUE

- ☒ You completed the data model plan.
- ☒ You are signed in to Supabase.
- ☒ The SQL Editor is open.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write one complete Supabase SQL script that creates my sales table.

Requirements:

- Name the table sales.
- Use id as a UUID primary key with a safe default.
- Use user_id as a required UUID linked to auth.users(id) with on delete cascade.
- Add sale_date as a required date.
- Add item_name as required text and category as required text.
- Add quantity as a required integer greater than zero.
- Add unit_price as a required numeric value that cannot be negative.
- Add total_amount as a generated stored numeric column equal to quantity multiplied by unit_price.
- Add payment_method as required text.
- Add optional customer_name and notes fields.
- Add created_at and updated_at timestamps with safe defaults.
- Create useful indexes for user_id, sale_date, category and payment_method.
- Use if not exists where it is safe.
- Explain exactly how to run the script in Supabase.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- One complete table-creation script.
- Database checks for quantity and price.
- A generated total and helpful indexes.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as sales-table.sql.

2.

Run the complete script in the Supabase SQL Editor.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open Table Editor and find sales.
- ☒ Check that every requested column exists.
- ☒ Confirm total_amount is generated by the database.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ The sales table exists.
- ☒ The owner, date, item, price and total fields are present.

COMMON BEGINNER MISTAKES

- Running only part of the SQL.
- Changing a column name in Supabase but not in your notes.
- Making total_amount a normal text field.

BEHIND THE SCENES

PostgreSQL can calculate a generated column every time quantity or unit price changes. This prevents the saved total from disagreeing with the numbers used to create it.

THINK LIKE A SOFTWARE DESIGNER

Why is a database-generated total safer than trusting a total typed into the browser?

WHAT YOU LEARNED

You learned:

- How to create the sales table.
- How database checks protect important numbers.
- How a generated total stays accurate.

LESSON 3

PROTECTING SALES VALUES

Estimated Time

25–35 Minutes

WHAT YOU ARE BUILDING

Rules that reject incomplete dates, blank names and unsupported payment methods.

WHY THIS MATTERS

The form helps users, but database rules protect the information even if a request does not come from the form.

BEFORE YOU CONTINUE

- ☒ The sales table exists.
- ☒ You can see its columns in Table Editor.
- ☒ You backed up sales-table.sql.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write one complete follow-up SQL script that strengthens the sales table.

Requirements:

- Reject blank `item_name` and blank category after spaces are removed.
- Allow only Cash, Card, Bank Transfer, Mobile Money or Other for `payment_method`.
- Keep quantity greater than zero and `unit_price` zero or greater.
- Create a trigger that updates `updated_at` whenever a row changes.
- Make the script safe to run after the table-creation script.
- Explain each rule in simple language.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- Complete SQL for validation rules.
- A complete `updated_at` trigger.
- Simple explanations.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as `sales-rules.sql`.

2.

Run the full script in the SQL Editor.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Try to add a test row with quantity 0 in Table Editor.
- ☒ Try an unsupported payment method.
- ☒ Confirm Supabase refuses invalid values.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Invalid quantities are rejected.
- ☒ Only the allowed payment methods are accepted.

COMMON BEGINNER MISTAKES

- Using different spelling in the form and database rule.
- Deleting earlier checks while adding new ones.
- Testing with a real business record instead of temporary test data.

BEHIND THE SCENES

Validation close to the database is a final safety net. JavaScript validation improves the experience; database validation protects the stored information.

THINK LIKE A SOFTWARE DESIGNER

Which payment-method value would be rejected, and which line of SQL causes that rejection?

WHAT YOU LEARNED

You learned:

- Why important rules belong in the database.
- How allowed values keep reports consistent.
- How updated_at records later changes.

LESSON 4

ENABLING ROW LEVEL SECURITY

Estimated Time

20–30 Minutes

WHAT YOU ARE BUILDING

Row Level Security on the sales table.

WHY THIS MATTERS

Without Row Level Security, a database request could expose sales that belong to another account.

BEFORE YOU CONTINUE

- ☒ The sales table and validation rules work.
- ☒ You understand that `user_id` identifies the owner.
- ☒ The SQL Editor is open.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write the complete SQL needed to enable Row Level Security on the sales table.

Requirements:

- Enable Row Level Security.
- Do not create public access.
- Explain why enabling RLS alone blocks browser access until policies are added.
- Include a query I can use to confirm that RLS is enabled.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- The RLS command.
- A confirmation query.
- A beginner-friendly explanation.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as sales-rls.sql.

2.

Run it in Supabase.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

☒ Confirm RLS shows as enabled for sales.

☒ Do not disable RLS to make a later test pass.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

☒ RLS is enabled.

☒ There is no public policy.

COMMON BEGINNER MISTAKES

- Assuming login alone protects database rows.
- Turning RLS off when a policy is missing.
- Adding a policy for the anon role.

BEHIND THE SCENES

Authentication identifies the signed-in user. Row Level Security then decides which rows that user may read or change.

THINK LIKE A SOFTWARE DESIGNER

What is the difference between knowing who a user is and deciding which rows that user may access?

WHAT YOU LEARNED

You learned:

- How to enable RLS.
- Why RLS starts with no access.
- Why policies must check ownership.

LESSON 5

CREATING THE SELECT POLICY

Estimated Time

20–30 Minutes

WHAT YOU ARE BUILDING

A policy that lets a signed-in user read only their own sales.

WHY THIS MATTERS

Private sales figures must never appear in another user's dashboard or reports.

BEFORE YOU CONTINUE

- ☒ RLS is enabled.
- ☒ You have not created public access.
- ☒ You know that `auth.uid()` represents the signed-in user.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write one complete SQL script for the sales SELECT policy.

Requirements:

- Allow only the authenticated role.
- Allow a row only when `auth.uid()` equals `user_id`.
- Drop and recreate the named policy safely.
- Use a clear policy name.
- Explain the ownership check in simple language.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A safe SELECT policy.
- An authenticated ownership check.
- A simple explanation.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as `sales-select-policy.sql`.

2.

Run the full script.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ✓ Open the policy list for sales.
 - ✓ Confirm the policy applies to SELECT and authenticated users.
- Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ✓ The SELECT policy exists.
- ✓ Its condition compares `auth.uid()` with `user_id`.

COMMON BEGINNER MISTAKES

- Using `true` as the policy condition.
- Applying the policy to `anon`.
- Comparing `auth.uid()` with the sale id.

BEHIND THE SCENES

A SELECT policy is applied whenever the application asks Supabase to read sales. Rows that fail the condition are hidden.

THINK LIKE A SOFTWARE DESIGNER

If a table contains ten rows but only four belong to the signed-in user, how many should Supabase return?

WHAT YOU LEARNED

You learned:

- How SELECT policies filter rows.
- How `auth.uid()` supports ownership.
- Why private totals depend on secure reads.

LESSON 6

CREATING THE INSERT POLICY

Estimated Time

20–30 Minutes

WHAT YOU ARE BUILDING

A policy that allows a user to save a sale only under their own account.

WHY THIS MATTERS

A browser must not be allowed to create a record and assign it to someone else.

BEFORE YOU CONTINUE

- ☒ The SELECT policy works.
- ☒ RLS remains enabled.
- ☒ The SQL Editor is open.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write one complete SQL script for the sales INSERT policy.

Requirements:

- Allow only authenticated users.
- Use WITH CHECK so `auth.uid()` must equal `user_id`.
- Drop and recreate the named policy safely.
- Explain why the application must insert the current user's id.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A secure INSERT policy.
- A WITH CHECK ownership rule.
- A simple explanation.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as `sales-insert-policy.sql`.

2.

Run the complete script.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Confirm the policy is for INSERT.
 - ☒ Confirm it uses WITH CHECK and the authenticated role.
- Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ The INSERT policy exists.
- ☒ A user cannot choose a different owner.

COMMON BEGINNER MISTAKES

- Using USING instead of WITH CHECK for INSERT.
- Forgetting the authenticated role.
- Using an email address as user_id.

BEHIND THE SCENES

WITH CHECK examines the new row before Supabase saves it. The ownership rule must be true for the insert to succeed.

THINK LIKE A SOFTWARE DESIGNER

Which value should your JavaScript place in user_id before inserting a sale?

WHAT YOU LEARNED

You learned:

- How INSERT policies protect ownership.
- Why WITH CHECK is required.
- Why the authenticated user id is saved with every sale.

LESSON 7

CREATING THE UPDATE POLICY

Estimated Time

20–30 Minutes

WHAT YOU ARE BUILDING

A policy that allows owners to edit only their own sales and keeps ownership unchanged.

WHY THIS MATTERS

Editing must not become a way to take over another user's row or transfer a row to a different account.

BEFORE YOU CONTINUE

- ☒ SELECT and INSERT policies exist.
- ☒ RLS remains enabled.
- ☒ The SQL Editor is open.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write one complete SQL script for the sales UPDATE policy.

Requirements:

- Allow only authenticated users.
- Use USING to check the existing row owner.
- Use WITH CHECK to check the updated row owner.
- Require auth.uid() to equal user_id in both checks.
- Drop and recreate the named policy safely.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A secure UPDATE policy.
- Both existing-row and new-row checks.
- A simple explanation.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as sales-update-policy.sql.

2.

Run the complete script.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

☒ Confirm the policy includes USING.

☒ Confirm it also includes WITH CHECK.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

☒ Only owners can update rows.

☒ An update cannot change ownership.

COMMON BEGINNER MISTAKES

- Adding only a USING condition.
- Allowing anon updates.
- Removing earlier policies.

BEHIND THE SCENES

An UPDATE has an old row and a proposed new row. Checking both protects the row before and after the change.

THINK LIKE A SOFTWARE DESIGNER

Why does a secure UPDATE policy need two ownership checks?

WHAT YOU LEARNED

You learned:

- How secure updates work.
- Why ownership is checked twice.
- How RLS protects editing.

LESSON 8

CREATING THE DELETE POLICY

Estimated Time

20–30 Minutes

WHAT YOU ARE BUILDING

A policy that permits permanent deletion only for the owner of a sale.

WHY THIS MATTERS

Delete is a powerful action. The database must reject attempts to delete another user's record.

BEFORE YOU CONTINUE

- ☒ The UPDATE policy exists.
- ☒ You understand that deletion is permanent.
- ☒ The SQL Editor is open.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write one complete SQL script for the sales DELETE policy.

Requirements:

- Allow only authenticated users.
- Use an ownership condition where `auth.uid()` equals `user_id`.
- Drop and recreate the named policy safely.
- Explain that the interface must still ask for confirmation before deleting.

Return the complete updated contents of:

- One complete SQL script

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A secure DELETE policy.
- An ownership check.
- A warning about permanent deletion.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the SQL as `sales-delete-policy.sql`.

2.

Run the complete script.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Confirm the policy applies to DELETE.
- ☒ Confirm it is restricted to authenticated owners.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ The DELETE policy exists.
- ☒ There is no public delete access.

COMMON BEGINNER MISTAKES

- Using a policy condition of true.
- Testing deletion with important data.
- Assuming a confirmation message replaces RLS.

BEHIND THE SCENES

The interface confirmation prevents accidental clicks. The DELETE policy prevents unauthorised database requests. Both protections have different jobs.

THINK LIKE A SOFTWARE DESIGNER

Why are a confirmation dialog and a DELETE policy both needed?

WHAT YOU LEARNED

You learned:

- How to protect deletion.
- Why ownership is required.
- Why user experience and database security work together.

LESSON 9

TESTING THE SALES DATABASE

Estimated Time

35–45 Minutes

WHAT YOU ARE BUILDING

A careful two-account test of all sales table rules and policies.

WHY THIS MATTERS

Security is not complete because the SQL ran successfully. You must prove that valid work succeeds and unauthorised work fails.

BEFORE YOU CONTINUE

- ☒ All four RLS policies exist.
- ☒ RLS is enabled.
- ☒ You can use two verified test accounts.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Create a beginner-friendly database security test plan for my sales table.

Requirements:

- Use the deployed HTTPS application and two separate verified accounts called Account A and Account B.
- Test valid inserts, reads, updates and deletes.
- Test invalid quantity, blank item name and unsupported payment method.
- Prove Account A cannot read, edit or delete Account B's rows.
- Do not suggest disabling RLS or using the service role key in browser code.
- Explain the expected result for every test.

Return the complete updated contents of:

- One complete test plan

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A numbered two-account test plan.
- Expected pass and fail results.
- A short security review.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the test plan in your project notes.

2.

Keep all SQL files together.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Create a temporary valid sale for Account A.
- ☒ Create a different temporary sale for Account B.
- ☒ Complete every ownership and validation test.
- ☒ Remove only temporary records when testing is complete.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Valid records succeed.
- ☒ Invalid values fail.
- ☒ Cross-account access fails.

COMMON BEGINNER MISTAKES

- Testing with only one account.
- Calling an empty result proof of every policy.
- Leaving RLS disabled after troubleshooting.

BEHIND THE SCENES

A two-account test checks the boundary between users. This is one of the most useful security tests in a multi-user application.

THINK LIKE A SOFTWARE DESIGNER

Which failed test would show that your SELECT policy is too open?

WHAT YOU LEARNED

You learned:

- How to test database rules.
- How to prove ownership isolation.
- Why expected failures are successful security tests.

CHAPTER SUMMARY

You created the secure database foundation for the Sales Tracker. The table stores useful sales information, calculates each row total and uses ownership policies to keep accounts separate.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ The sales table exists.
- ☒ Invalid quantities, prices and payment methods are rejected.
- ☒ The four ownership policies exist.
- ☒ Two-account privacy tests pass.

TRANSITION

The database is ready. In the next chapter, you will build the private dashboard and record the first real sale.

CHAPTER 5

BUILDING THE SALES DASHBOARD

CHAPTER INTRODUCTION

The secure database is ready.

You will now create the main working area of the application.

The user will record a sale, see its calculated total and view useful daily and monthly figures. Each step will be tested before you move forward.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A protected sales dashboard
- A complete Record Sale form
- Secure sale saving
- Daily, monthly and all-time statistics

LESSON 1

DESIGNING THE SALES DASHBOARD

Estimated Time

35–45 Minutes

WHAT YOU ARE BUILDING

The private dashboard layout, navigation and empty statistics cards.

WHY THIS MATTERS

The dashboard gives the user one clear place to understand sales and begin common tasks.

BEFORE YOU CONTINUE

- ☒ Authentication works on the deployed site.
- ☒ The sales table is secure.
- ☒ You backed up your project folder.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Build the complete private Sales Tracker dashboard.

Requirements:

- Protect the page so signed-out visitors go to login.html.
- Show the signed-in user's email and a working logout button.
- Create cards for today's sales total, today's sale count, this month's sales total and all-time sales total.
- Create a clear Record New Sale button.
- Create links to Dashboard and Sales History.
- Show friendly loading and empty states.
- Use accessible labels, visible keyboard focus and responsive design.
- Do not calculate real statistics yet; prepare the elements with clear ids.

Return the complete updated contents of:

- dashboard.html
- dashboard.js
- styles.css

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A protected dashboard.
- Four prepared statistics cards.
- Clear navigation and responsive styling.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Replace dashboard.html.

2.

Replace dashboard.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open the dashboard while signed in.
- ☒ Test the navigation and logout button.
- ☒ Resize the browser and check the cards.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Signed-out users are redirected.
- ☒ The dashboard is clear on desktop and mobile.
- ☒ No unexpected console errors appear.

COMMON BEGINNER MISTAKES

- Removing existing authentication checks.

- Using the same id on several cards.
- Showing zero before loading finishes without a loading state.

BEHIND THE SCENES

The HTML provides labelled spaces for information. JavaScript will later ask Supabase for sales and place calculated values inside those spaces.

THINK LIKE A SOFTWARE DESIGNER

Which dashboard number should help a business understand today's activity most quickly, and why?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to organise a private dashboard.
- How to prepare statistic elements.
- Why loading and empty states matter.

LESSON 2

BUILDING THE RECORD SALE FORM

Estimated Time

40–50 Minutes

WHAT YOU ARE BUILDING

A complete form for entering one sale.

WHY THIS MATTERS

A good form helps the user provide valid information without needing to understand the database.

BEFORE YOU CONTINUE

- ☒ The dashboard works.
- ☒ You know the sales table field names.
- ☒ You backed up the current files.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Add a complete Record Sale form to the protected dashboard.

Requirements:

- Include sale date, item name, category, quantity, unit price, payment method, optional customer name and optional notes.
- Use today's date as a helpful default without preventing an earlier date.
- Use a select for the allowed payment methods.
- Show a live read-only total calculated from quantity multiplied by unit price.
- Use decimal-safe display formatting for money.
- Add clear labels, help text and validation messages.
- Do not save to Supabase yet.
- Keep all existing dashboard and authentication features.

Return the complete updated contents of:

- dashboard.html
- dashboard.js
- styles.css

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete accessible form.
- A live calculated total.
- Clear validation and mobile styling.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Replace dashboard.html.

2.

Replace dashboard.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Enter quantity 3 and unit price 2500.
- ☒ Confirm the displayed total is 7500 in the chosen money format.
- ☒ Try blank required fields and quantity 0.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ All requested fields appear.
- ☒ The total updates immediately.
- ☒ Invalid form values are explained clearly.

COMMON BEGINNER MISTAKES

- Using a text field for quantity.

- Allowing a negative price.
- Saving a formatted currency string instead of a number.

BEHIND THE SCENES

The browser displays a helpful total before saving. The database will calculate its own generated total after saving, so the stored value remains trustworthy.

THINK LIKE A SOFTWARE DESIGNER

Which two input values cause the live total to change?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to build a sales form.
- How to calculate a preview total.
- How form controls reduce errors.

LESSON 3

SAVING SALES SECURELY

Estimated Time

40–50 Minutes

WHAT YOU ARE BUILDING

The complete form submission that saves a sale under the signed-in user's account.

WHY THIS MATTERS

This is the moment the form becomes a working business capability instead of a visual design.

BEFORE YOU CONTINUE

- ☒ The form validates correctly.
- ☒ The INSERT policy exists.
- ☒ You are using the publishable key, never a service role key.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Connect the Record Sale form to Supabase and save sales securely.

Requirements:

- Get the authenticated user before inserting.
- Insert `sale_date`, `item_name`, `category`, `quantity`, `unit_price`, `payment_method`, `customer_name`, `notes` and the current user's id.
- Do not insert `total_amount` because the database generates it.
- Trim text values and convert number inputs to numbers.
- Disable the submit button while saving.
- Show a clear success or error message.
- After success, reset the form, restore today's date and refresh dashboard data.
- Keep all existing features.

Return the complete updated contents of:

- `dashboard.html`
- `dashboard.js`
- `styles.css`

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete secure insert flow.
- Duplicate-click protection.
- Clear success and error feedback.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Replace dashboard.html.

2.

Replace dashboard.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Save a valid sale.
- ☒ Confirm the row appears in Supabase with your user_id.
- ☒ Confirm total_amount equals quantity multiplied by unit_price.
- ☒ Refresh the page and confirm the record remains.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ A valid sale saves once.
- ☒ The generated total is correct.
- ☒ An error does not clear the form.

COMMON BEGINNER MISTAKES

- Adding total_amount to the insert.
- Forgetting to await Supabase.
- Clearing the form before Supabase confirms success.

BEHIND THE SCENES

The submit handler collects values, validates them, identifies the user and sends one insert request. RLS makes the ownership check again inside the database.

THINK LIKE A SOFTWARE DESIGNER

Find the insert object. Which property connects the new row to the signed-in account?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to save a sale.
- Why the generated total is not inserted.
- How to handle loading, success and failure.

LESSON 4

BUILDING DAILY AND MONTHLY STATISTICS

Estimated Time

45–60 Minutes

WHAT YOU ARE BUILDING

Real dashboard statistics calculated from the signed-in user's sales.

WHY THIS MATTERS

Daily and monthly figures help a business see current performance without reading every row.

BEFORE YOU CONTINUE

- ☒ You saved several test sales on different dates.
- ☒ The SELECT policy works.
- ☒ The dashboard statistic elements have unique ids.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Load the signed-in user's sales and calculate the dashboard statistics.

Requirements:

- Select only the fields needed for the statistics.
- Use `sale_date`, not `created_at`, to decide which period contains a sale.
- Calculate today's total and today's number of sales.
- Calculate the current calendar month's total.
- Calculate the all-time total.
- Use local date parts carefully so today's date is not shifted by UTC conversion.
- Format money consistently.
- Show loading, empty and error states.
- Refresh the statistics after a new sale is saved.

Return the complete updated contents of:

- `dashboard.html`
- `dashboard.js`
- `styles.css`

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- Complete statistics-loading logic.
- Correct daily, monthly and all-time calculations.
- Friendly states and formatted values.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Replace dashboard.html.

2.

Replace dashboard.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

☒ Add two sales dated today and one dated earlier this month.

☒ Check today's count and total.

☒ Check the monthly and all-time totals.

☒ Change one test date and confirm the correct cards change.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

☒ Every card matches the saved test data.

☒ The figures survive refresh.

☒ Only the signed-in user's rows are included.

COMMON BEGINNER MISTAKES

- Using `created_at` instead of `sale_date`.
- Comparing dates after converting them to UTC.
- Counting quantity as the number of sales when the card is meant to count rows.

BEHIND THE SCENES

The browser receives the owner's permitted rows, then uses filter and reduce operations to group dates and add totals. RLS ensures the input data already belongs to the current account.

THINK LIKE A SOFTWARE DESIGNER

Which condition decides whether a sale belongs to the current month?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to calculate daily statistics.
- How to calculate monthly statistics.
- How date choices affect business reports.

CHAPTER SUMMARY

You built the central working area of the Sales Tracker. A signed-in user can now record a sale and immediately see useful totals.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ The form saves valid sales.
- ☒ The database generates correct totals.
- ☒ Daily and monthly statistics are accurate.
- ☒ The dashboard works on a small screen.

TRANSITION

The dashboard shows a useful summary. Next, you will build the complete sales history and a page for viewing one sale.

CHAPTER 6

VIEWING SALES

CHAPTER INTRODUCTION

The dashboard gives a quick summary, but users also need to see the records behind those figures.

You will create a complete Sales History page and a separate Sale Details page. Both pages will remain protected by authentication and Row Level Security.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A complete Sales History page
- Responsive sales rows and cards
- A Sale Details page
- Empty, missing and privacy tests

LESSON 1

BUILDING THE SALES HISTORY

Estimated Time

45–60 Minutes

WHAT YOU ARE BUILDING

A protected page that lists all saved sales in a clear order.

WHY THIS MATTERS

A business needs to move from a summary to the individual records behind it.

BEFORE YOU CONTINUE

- ☒ Several test sales exist.
- ☒ The SELECT policy works.
- ☒ Dashboard navigation works.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Build a complete protected Sales History page.

Requirements:

- Load the signed-in user's sales from Supabase.
- Order records by `sale_date` newest first, then `created_at` newest first.
- Show date, item, category, quantity, unit price, total and payment method.
- Use a responsive table on wide screens and readable cards on small screens.
- Make each record open `sale-details.html?id=SALE_ID`.
- Add loading, error and no-sales states.
- Add navigation back to the dashboard.

Return the complete updated contents of:

- `sales.html`
- `sales.js`
- `styles.css`

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete protected history page.
- Responsive sales records.
- Working details links and useful states.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Create sales.html.

2.

Create sales.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open Sales History while signed in.
- ☒ Confirm all your records appear in the correct order.
- ☒ Open the page on a narrow browser window.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Only the current user's sales appear.
- ☒ Every amount and date is readable.
- ☒ Each details link contains the correct id.

COMMON BEGINNER MISTAKES

- Forgetting the authentication guard.

- Displaying raw unformatted numbers.
- Creating links without the sale id.

BEHIND THE SCENES

The page makes one ordered SELECT request. JavaScript then creates a safe visual row or card for each returned record.

THINK LIKE A SOFTWARE DESIGNER

Which part of the Supabase query controls the order of the history?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to load a sales history.
- How to display the same data responsively.
- How a record id connects list and details pages.

LESSON 2

BUILDING THE SALE DETAILS PAGE

Estimated Time

40–50 Minutes

WHAT YOU ARE BUILDING

A protected page that shows every saved detail for one sale.

WHY THIS MATTERS

A focused details page gives the user space to review one record before editing or deleting it.

BEFORE YOU CONTINUE

- ☒ Sales History works.
- ☒ Its links contain an id query value.
- ☒ You backed up the project.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Build the complete protected sale details page.

Requirements:

- Read the id value from the page URL.
- Reject a missing id with a clear message and a link back.
- Select one sale by id using maybeSingle.
- Rely on RLS and also handle a missing or inaccessible record safely.
- Show all sale fields with formatted date and money values.
- Add Edit Sale, Delete Sale and Back to Sales History actions.
- Do not build editing or deletion yet.
- Add loading and error states.

Return the complete updated contents of:

- sale-details.html
- sale-details.js
- styles.css

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete protected details page.
- Safe id handling.
- Prepared edit and delete actions.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Create sale-details.html.

2.

Create sale-details.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open a valid sale from history.
- ☒ Remove the id from the URL and check the message.
- ☒ Use a random id and confirm the page fails safely.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ A valid sale displays correctly.
- ☒ Missing records do not crash the page.
- ☒ Navigation returns to the history.

COMMON BEGINNER MISTAKES

- Using the id without checking it.

- Calling single when a missing row should be handled normally.
- Showing raw database error details to the user.

BEHIND THE SCENES

The query-string id tells the page which row to request. RLS remains the final decision-maker about whether that row is visible.

THINK LIKE A SOFTWARE DESIGNER

What should the page do when maybeSingle returns no sale?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to read a URL parameter.
- How to load one protected row.
- How to handle missing records safely.

LESSON 3

TESTING THE SALES HISTORY

Estimated Time

30–40 Minutes

WHAT YOU ARE BUILDING

A complete test of list, details, empty, refresh and privacy behaviour.

WHY THIS MATTERS

A screen is only complete when normal results and unusual results are both understandable.

BEFORE YOU CONTINUE

- ☒ The history and details pages are complete.
- ☒ You have two verified test accounts.
- ☒ You can open browser developer tools.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write a complete beginner-friendly test checklist for Sales History and Sale Details.

Requirements:

- Test order, displayed values, details links, refresh and mobile layout.
- Test no-sales, missing-id and unknown-id states.
- Use two accounts to confirm a sale from Account A cannot be opened by Account B.
- Include the expected result for each step.
- Include a console-error check.

Return the complete updated contents of:

- One complete test checklist

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A normal-flow checklist.
- Empty and error-state tests.
- A two-account privacy test.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the checklist in your project notes.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Complete every checklist item.

☒ Record and fix each unexpected result.

☒ Repeat a failed test after the fix.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

☒ History and details work after refresh.

☒ Private records remain private.

☒ No unexplained console errors remain.

COMMON BEGINNER MISTAKES

- Testing only the first record.
- Skipping the mobile layout.
- Sharing one browser session between two account tests without signing out.

BEHIND THE SCENES

Testing different states helps you find problems that perfect sample data can hide.

THINK LIKE A SOFTWARE DESIGNER

Which test proves that RLS protects the details page?

WHAT YOU LEARNED

You learned:

- How to test connected pages.
- How to test empty and missing states.
- How to confirm cross-account privacy.

CHAPTER SUMMARY

You built a clear path from the dashboard to the complete history and then to one individual sale.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ Sales appear in the correct order.
- ☒ One sale opens safely by id.
- ☒ Empty and missing states are clear.
- ☒ Account privacy tests pass.

TRANSITION

The history works, but a growing list can become difficult to use. Next, you will add search, filters and sorting.

CHAPTER 7

SEARCHING, FILTERING AND SORTING SALES

CHAPTER INTRODUCTION

As the business records more sales, scrolling becomes slower and less useful.

In this chapter, you will help the user find a sale by words, category, payment method or date. You will also let the user change the order of the results.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Sales search
- Category and payment filters
- A sales date range
- Four sorting choices
- Clear filters and result feedback

LESSON 1

BUILDING SALES SEARCH, FILTERING AND SORTING

Estimated Time

50–65 Minutes

WHAT YOU ARE BUILDING

One discovery toolbar that helps users find the right sales quickly.

WHY THIS MATTERS

Search and filters turn a long history into useful business information.

BEFORE YOU CONTINUE

- ☒ Sales History loads correctly.
- ☒ Your test data contains different dates, categories and payment methods.
- ☒ You backed up the files.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Add complete search, filtering and sorting to Sales History.

Requirements:

- Add one search field for item name, customer name and notes.
- Add category and payment-method filters.
- Add From Date and To Date filters based on sale_date.
- Add sorting for newest, oldest, highest total and lowest total.
- Allow all controls to work together.
- Use safe Supabase query methods and escape search input used in an or filter.
- Do not filter by user_id in place of RLS; keep RLS enabled.
- Add a Clear Filters button and a result count.
- Show a specific no-matching-sales message.
- Keep details links and responsive display working.

Return the complete updated contents of:

- sales.html
- sales.js
- styles.css

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete discovery toolbar.
- Combined query behaviour.
- Clear filters, result count and no-match feedback.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Replace sales.html.

2.

Replace sales.js.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Search for part of an item name.
- ☒ Combine a date range with a payment method.
- ☒ Sort matching records by highest total.
- ☒ Clear every control.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Controls work separately and together.
- ☒ Clear Filters restores the full history.
- ☒ No-match feedback is different from no saved sales.

COMMON BEGINNER MISTAKES

- Filtering only the currently visible page by accident.
- Using created_at for the date range.
- Building raw query strings from unchecked input.

BEHIND THE SCENES

Each control contributes a condition or ordering instruction to the query. Rebuilding the query when controls change keeps the displayed records consistent.

THINK LIKE A SOFTWARE DESIGNER

Which lines add the From Date and To Date conditions to the Supabase query?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to combine search and filters.
- How to sort totals and dates.
- Why clear feedback helps users understand results.

LESSON 2

TESTING SALES DISCOVERY

Estimated Time

30–40 Minutes

WHAT YOU ARE BUILDING

A planned test using known sales that proves every discovery control works together.

WHY THIS MATTERS

Combined controls can fail in ways that single-control tests do not reveal.

BEFORE YOU CONTINUE

- ☒ The discovery toolbar is complete.
- ☒ You have a small set of sales with known differences.
- ☒ You know the expected results.

BACK UP BEFORE REPLACING COMPLETE FILES

- Before you replace a file, make a copy of your current project folder.
- Add the words Before This Lesson and today's date to the copied folder name.
- If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

- I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.
- I use Notepad, so explain any manual step in simple language.
- Create a complete test plan for my Sales History search, filters and sorting.

Requirements:

- Start by defining six useful test sales.
- Test partial item search, customer search and notes search.
- Test category, payment method and date range separately.
- Test at least three combined-control cases.
- Test all four sorting options.
- Test Clear Filters, no matches, refresh and mobile layout.
- State the expected records for every test.

Return the complete updated contents of:

- One complete test plan

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A six-record test dataset.
- Individual and combined tests.
- Expected results for every test.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the plan in your project notes.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ✓ Create the planned test records.
- ✓ Complete every test.
- ✓ Correct and repeat any failed case.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ✓ Every control produces the expected records.
- ✓ Sorting changes order without losing filters.
- ✓ Clear Filters restores the list.

COMMON BEGINNER MISTAKES

- Testing with records that all look alike.
- Guessing expected totals during the test.
- Ignoring no-match and refresh behaviour.

BEHIND THE SCENES

Known test data makes failures easier to recognise. It gives you a small answer key for every filter combination.

THINK LIKE A SOFTWARE DESIGNER

Which combined test would reveal that changing the sort accidentally removes the date filter?

WHAT YOU LEARNED

You learned:

- How to design useful search tests.
- How to test combined state.
- Why expected results should be written first.

CHAPTER SUMMARY

You turned Sales History into a useful discovery tool. Users can combine controls and still open the correct record.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ Search finds the correct text.
- ☒ Filters work alone and together.
- ☒ All sorting options work.
- ☒ No-match and clear-filter states are correct.

TRANSITION

Users can now find the right sale. Next, you will allow them to correct a saved record securely.

CHAPTER 8

EDITING SALES

CHAPTER INTRODUCTION

A saved sale may contain a typing mistake or need a correction.

You will build one focused edit page. It will reuse familiar form rules, protect the row with RLS and allow the database to calculate the revised total.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A protected Edit Sale page
- Prefilled sale information
- Live revised totals
- Secure saving and ownership tests

LESSON 1

BUILDING SALE EDITING

Estimated Time

50–65 Minutes

WHAT YOU ARE BUILDING

A complete edit page that loads one sale, validates changes and saves them securely.

WHY THIS MATTERS

Mistakes happen. Editing lets the owner correct a date, item, quantity, price or other detail without creating a second record.

BEFORE YOU CONTINUE

- ☒ Sale Details works.
- ☒ The UPDATE policy exists.
- ☒ You backed up the project.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Build complete secure editing for one sale.

Requirements:

- Create edit-sale.html and edit-sale.js.
- Read the sale id from the URL and protect the page with authentication.
- Load the owner's sale and prefill every editable field.
- Reuse the same field rules and live total preview as Record Sale.
- Update only sale_date, item_name, category, quantity, unit_price, payment_method, customer_name and notes.
- Never update user_id, id, total_amount or created_at from the browser.
- Disable the save button while updating.
- Show clear success and error messages.
- After success, return to the same sale details page.
- Connect the Sale Details Edit button to this page.

Return the complete updated contents of:

- edit-sale.html
- edit-sale.js
- sale-details.html
- sale-details.js
- styles.css

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete protected edit page.
- Prefilled values and recalculated preview.
- A secure update and return flow.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Create edit-sale.html.

2.

Create edit-sale.js.

3.

Replace the complete details files returned by AI.

4.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open Edit from one sale.
- ☒ Change quantity and unit price.
- ☒ Save and confirm the details page shows the new generated total.
- ☒ Refresh and confirm the change remains.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ All current values are prefilled.
- ☒ Only allowed fields are updated.
- ☒ The generated total changes correctly.

COMMON BEGINNER MISTAKES

- Sending `user_id` in the update object.
- Allowing edits before the existing row loads.
- Calculating a preview but not validating the number inputs.

BEHIND THE SCENES

The page first reads the protected row, then sends only allowed changes. The database recalculates `total_amount` and the UPDATE policy verifies ownership.

THINK LIKE A SOFTWARE DESIGNER

Which fields are deliberately missing from the update object, and why?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to prefill an edit form.
- How to update only allowed fields.
- How generated totals respond to changes.

LESSON 2

TESTING SALE EDITING

Estimated Time

30–40 Minutes

WHAT YOU ARE BUILDING

A complete test of valid edits, invalid edits, totals, refresh and ownership.

WHY THIS MATTERS

Editing touches existing business data, so it deserves careful tests before the feature is trusted.

BEFORE YOU CONTINUE

- ☒ Editing works for one normal test.
- ☒ You have two test accounts.
- ☒ Your Error Log is ready.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write a complete beginner-friendly test plan for sale editing.

Requirements:

- Test every editable field.
- Test quantity and unit-price changes and the generated total.
- Test blank required fields, quantity zero and a negative price.
- Test missing and unknown ids.
- Test refresh after saving.
- Prove Account B cannot edit Account A's sale.
- Include expected results and a console check.

Return the complete updated contents of:

- One complete test plan

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- Valid and invalid edit tests.
- A total recalculation test.
- A two-account security test.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the plan in your project notes.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ✓ Complete every test.
- ✓ Check the database row after important changes.
- ✓ Repeat failed tests after fixing them.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ✓ Valid edits persist.
- ✓ Invalid edits do not damage the row.
- ✓ Cross-account editing fails.

COMMON BEGINNER MISTAKES

- Testing only one field.
- Not checking the database-generated total.
- Using a record that both test accounts can legitimately access.

BEHIND THE SCENES

A secure failure may appear as no accessible row or no updated row. The important result is that another account's data remains unchanged.

THINK LIKE A SOFTWARE DESIGNER

Which test proves that changing quantity updates the saved total rather than only the preview?

WHAT YOU LEARNED

You learned:

- How to test existing-data changes.
- How to check generated totals after edits.
- How to confirm update privacy.

CHAPTER SUMMARY

You built and tested secure sale editing. The owner can correct useful fields without changing the identity or ownership of the record.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ Existing values load correctly.
- ☒ Valid changes persist.
- ☒ Invalid changes are refused.
- ☒ Another account cannot edit the sale.

TRANSITION

The application can create, view, find and edit sales. Next, you will add careful permanent deletion.

CHAPTER 9

DELETING SALES

CHAPTER INTRODUCTION

The final record-management capability is deletion.

You will make the decision clear, give the user a chance to cancel and rely on Row Level Security to protect ownership. Use temporary records while testing this chapter.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A clear delete confirmation
- A protected delete request
- Success feedback
- Connected-screen and two-account tests

LESSON 1

BUILDING SECURE SALE DELETION

Estimated Time

40–50 Minutes

WHAT YOU ARE BUILDING

A clear confirmation flow that permanently deletes one owned sale.

WHY THIS MATTERS

Deletion is useful for duplicate or unwanted records, but it must be deliberate and protected.

BEFORE YOU CONTINUE

- ☒ The DELETE policy exists.
- ☒ Sale Details loads the correct record.
- ☒ You created a temporary record for deletion testing.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Add complete secure deletion to the Sale Details page.

Requirements:

- Use the current sale id.
- Show a clear confirmation that includes the item name and says the action cannot be undone.
- Do nothing when the user cancels.
- Disable the delete control while the request runs.
- Delete by id and rely on the RLS ownership policy.
- Show a clear error if deletion fails.
- After success, go to sales.html with a simple deleted-successfully message.
- Show that message on Sales History and then remove it from the URL without reloading.
- Keep all view and edit features working.

Return the complete updated contents of:

- sale-details.html
- sale-details.js
- sales.html
- sales.js
- styles.css

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A deliberate confirmation.
- A secure delete request.
- A success return to Sales History.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Confirm that you made a separate copy of the complete working project folder before replacing any existing file.

1.

Replace the complete details files.

2.

Replace the complete history files.

3.

Replace styles.css.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Open a temporary sale and choose Delete.
- ☒ Cancel once and confirm the row remains.
- ☒ Confirm again and verify the row is removed.
- ☒ Check that the history success message appears.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Cancellation changes nothing.
- ☒ Confirmation deletes exactly one owned sale.
- ☒ Statistics change after returning to the dashboard.

COMMON BEGINNER MISTAKES

- Using a vague confirmation message.
- Deleting before the user confirms.
- Leaving the button active during the request.

BEHIND THE SCENES

The browser asks for a deliberate decision, then sends one DELETE request. RLS checks the row owner before PostgreSQL removes anything.

THINK LIKE A SOFTWARE DESIGNER

Where does the code stop when the user cancels the confirmation?

CODE-READING QUESTION

In the complete JavaScript or HTML from this lesson, find the part that performs the main action. What information does it use, and what happens when the action succeeds?

WHAT YOU LEARNED

You learned:

- How to build a safe delete flow.
- How RLS protects deletion.
- How to return clear feedback after deletion.

LESSON 2

TESTING SALE DELETION

Estimated Time

30–40 Minutes

WHAT YOU ARE BUILDING

A complete deletion test covering cancellation, confirmation, totals, missing records and privacy.

WHY THIS MATTERS

Because deletion cannot be undone, the feature must behave correctly in both success and failure cases.

BEFORE YOU CONTINUE

- ☒ Secure deletion is implemented.
- ☒ You have disposable test sales.
- ☒ You have two verified accounts.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Write a complete beginner-friendly test plan for sale deletion.

Requirements:

- Use only disposable test sales.
- Test cancelling the confirmation.
- Test confirming deletion.
- Confirm the deleted sale disappears from history, details and dashboard totals.
- Test refreshing the old details URL.
- Prove Account B cannot delete Account A's sale.
- Include expected results and an unexpected-error check.

Return the complete updated contents of:

- One complete test plan

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- Safe disposable test preparation.
- Success and cancellation tests.
- A two-account privacy test.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the test plan in your project notes.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ✓ Complete every test with temporary data.
- ✓ Confirm database rows directly when needed.
- ✓ Record and fix unexpected results.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ✓ Cancel keeps the row.
- ✓ Confirm removes only the selected row.
- ✓ Cross-account deletion fails.
- ✓ Dashboard totals update correctly.

COMMON BEGINNER MISTAKES

- Deleting an important record.
- Checking only the screen and not the database.
- Forgetting to retest totals after deletion.

BEHIND THE SCENES

Deletion affects every screen that used the record. A complete test checks the database, history, details and summary figures.

THINK LIKE A SOFTWARE DESIGNER

Which screens should change after a sale is deleted, and what should each one show?

WHAT YOU LEARNED

You learned:

- How to test permanent actions safely.
- How deletion affects connected features.
- How to prove delete ownership.

CHAPTER SUMMARY

You added deliberate, secure deletion and confirmed that related lists and statistics respond correctly.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ Cancelling keeps the sale.
- ☒ Confirming removes one selected sale.
- ☒ Only the owner can delete.
- ☒ Related figures update correctly.

TRANSITION

Every required sales capability is now present. The final chapter brings the complete journey together and prepares the application for sharing.

CHAPTER 10

FINAL APPLICATION REVIEW

CHAPTER INTRODUCTION

You have built every major part of the Sales Tracker.

This chapter is about confidence. You will test the complete journey, publish the latest version and prepare a simple description for your portfolio.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- One end-to-end test
- A final security review
- A live deployment check
- A portfolio description and reflection

LESSON 1

COMPLETE SYSTEM TESTING

Estimated Time

60–90 Minutes

WHAT YOU ARE BUILDING

One end-to-end test of the complete Sales Tracker on its deployed HTTPS website.

WHY THIS MATTERS

Separate features may work on their own and still fail when used as one complete journey.

BEFORE YOU CONTINUE

- ☒ Every chapter milestone is complete.
- ☒ Your latest project folder is backed up.
- ☒ You have two verified test accounts.

BACK UP BEFORE REPLACING COMPLETE FILES

Before you replace a file, make a copy of your current project folder.

Add the words Before This Lesson and today's date to the copied folder name.

If something goes wrong, you can return to the working copy.

BUILD PROMPT

PROMPT STARTS HERE

I am building a beginner-friendly Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

I use Notepad, so explain any manual step in simple language.

Create one complete final test plan for my beginner Sales Tracker.

Requirements:

- Test registration, email verification, login, protected pages and logout on the deployed HTTPS Netlify website.
- Test recording sales and generated totals.
- Test daily, monthly and all-time dashboard statistics.
- Test history, details, search, filters and sorting.
- Test editing and deletion.
- Test empty, loading, error and mobile states.
- Use two accounts to test read, insert, update and delete ownership.
- Include expected results and a final browser-console check.

Return the complete updated contents of:

- One complete end-to-end test plan

Do not return snippets.

Do not use frameworks, package managers or build tools.

Keep the existing design and working features unless a change is required for this task.

After the files, give me a short beginner-friendly testing checklist.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return:

- A complete ordered test journey.
- Expected results for every major capability.
- A final security and responsive review.

Do not continue if AI returns only a snippet or leaves out a file that it was asked to update. Ask for the complete file again.

SAVE YOUR FILES

1.

Save the final test plan in your project notes.

2.

Save every complete current project file.

Use Save As when creating a new file. Choose All Files and UTF-8 if Notepad shows those options.

TEST YOUR WORK

- ☒ Complete the full journey with a new verified account.
- ☒ Repeat the ownership tests with a second account.
- ☒ Fix and repeat every failed test.
- ☒ Confirm there are no unexplained console errors.

Write any unexpected result in your Error Log before asking AI for help.

CHECKPOINT

Do not continue until:

- ☒ Every required capability works together.
- ☒ Saved data survives refresh and sign-in.
- ☒ Cross-account access fails.
- ☒ The application works on a small screen.

COMMON BEGINNER MISTAKES

- Testing only with an old browser session.
- Skipping expected failure tests.
- Publishing a change without repeating the affected tests.

BEHIND THE SCENES

End-to-end testing follows the same path a real user will follow. It checks the connections between authentication, database security, screens and business calculations.

THINK LIKE A SOFTWARE DESIGNER

Which three tests would you repeat first after changing the sales table?

WHAT YOU LEARNED

You learned:

- How to test a complete application.
- How to combine business and security checks.
- How to decide whether the project is ready to share.

CHAPTER SUMMARY

You completed the final application review. The Sales Tracker now supports secure sales recording, reporting, discovery, editing and deletion.

CHAPTER MILESTONE

Before moving forward, confirm:

- ☒ The complete live journey works.
- ☒ Daily and monthly figures are correct.
- ☒ Two-account privacy tests pass.
- ☒ No unexplained browser errors remain.

TRANSITION

Complete the final sections below before marking Workbook 06 as finished.

FINAL TESTING

Create a completely new user account on the deployed HTTPS website.

Complete this journey:

- ☒ Register and verify the email address.
- ☒ Sign in and confirm the empty states.
- ☒ Record sales on today, an earlier date this month and an earlier month.
- ☒ Check each generated total.
- ☒ Check today's, this month's and all-time dashboard totals.
- ☒ Open Sales History and one Sale Details page.
- ☒ Search, filter and sort the records.
- ☒ Edit one sale and confirm its revised total.
- ☒ Cancel one deletion, then delete a disposable sale.
- ☒ Refresh the website and confirm the remaining data still exists.
- ☒ Sign out and confirm protected pages redirect to login.
- ☒ Use a second account and confirm that the first account's sales remain private.

If a step fails, write the result in the Error Log, fix the problem and repeat the test.

GOING LIVE

Your Sales Tracker is ready for its final deployment.

1.

Save every complete project file.

2.

Make one final backup of the complete project folder.

3.

Open <https://app.netlify.com/drop>.

4.

Drag the complete Sales Tracker folder into the deployment area. If you already created the Netlify website during authentication testing, deploy the updated folder to that same website.

5.

Wait for deployment to finish and open the HTTPS website supplied by Netlify.

6.

Confirm that the Supabase Site URL and Redirect URLs use that exact HTTPS address.

7.

Repeat registration, sales recording, reports, history, search, editing, deletion and two-account privacy tests on the live website.

Only share the application after every important live test succeeds.

PORTFOLIO DESCRIPTION

Sales Tracker

I built a complete browser-based Sales Tracker using HTML, CSS, Vanilla JavaScript and Supabase.

The application allows authenticated users to record sales, calculate totals, monitor daily and monthly performance, search and filter sales, open individual records, edit sales and delete unwanted records.

I used Supabase Authentication and Row Level Security so each user can access only their own sales.

I also tested responsive design, database validation, generated totals and two-account privacy.

REFLECTION QUESTIONS

1.

Which part of the Sales Tracker are you most proud of?

2.

How does the database-generated total protect accuracy?

3.

What is the difference between today's total and this month's total?

4.

How does Row Level Security protect sales records?

5.

Which test helped you find the most useful problem?

6.

What would you improve before giving this application to a real business?

EXTENSION CHALLENGES

Complete these only after the main application works correctly.

- Add a best-selling categories summary.
- Add an export-to-CSV feature for the signed-in user's filtered sales.
- Add a simple printable monthly sales report.
- Add a setting for the business's preferred currency symbol.
- Add pagination when the sales history becomes long.

Ask AI for complete updated files, make a backup first and repeat the affected security and calculation tests after every extension.

NEXT WORKBOOK

Excellent work.

You have completed Workbook 06 of the Prompt to Profit™ Software Workbook Series.

You have built software that allows a business to record, review, search, edit and delete sales while monitoring daily and monthly totals.

The next project in the series is Workbook 07 — Supplier Management System.

For now, review what you have built and make sure every important feature works correctly.

Congratulations once again on completing your Sales Tracker.